Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 «Информатика и вычислительная техника»

FINAL QUALIFICATION WORK
(BACHELOR'S THESIS)
Academic Program
09.03.01 «Computer Science»

Направленность (профиль) образовательной программы
«Анализ данных и искусственный интеллект»

Field of Study:
«Data Science and Artificial Intelligence»

Тема

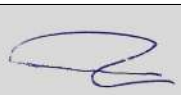
Устранение разрыва в проверке обеспечения безопасности: обнаружение расхождений между заявленным и фактическим положением безопасности в контейнерных средах Kubernetes

Topic

Bridging the Enforcement Verification Gap: Detecting Divergence between declared and actual security posture in kubernetes containerized environments

Работу выполнил / *Prepared by*

Собех Йехия / Yehia Sobeh



Руководитель выпускной квалификационной работы / *Final Qualification Work Supervisor*

Кирилл Салтанов / Kirill Saltanov



Иннополис, Innopolis, 2026

Contents

1 Introduction **11**

1.1 Relevance of the Topic (Background and Motivation) 11

1.2 Problem Statement 13

1.3 Research Questions 14

1.4 Aim of the Study 15

1.5 Tasks (Objectives) of the Study 15

1.6 Object and Subject of the Study 16

1.7 Scientific Novelty 16

1.8 Contributions 17

1.9 Thesis Structure 18

2 Literature Review **19**

2.1 Kubernetes Security Model 19

2.2 Seccomp Profiles in Kubernetes 21

2.2.1 The Three Seccomp Profile Types 22

2.2.2 Why Seccomp Matters for This Thesis 23

2.3 Static Misconfiguration Detection 24

2.4 Admission-Time Policy Enforcement 25

2.5 Runtime Behavioural Monitoring 27

2.6 Configuration Drift Detection 29

2.7 The Identified Gap 31

3 Design and Methodology **33**

3.1 Architectural Overview 33

3.2 Layer 1: Specification-Level Policy Checks 36

3.3 Layer 2: Runtime Drift Detection 37

3.3.1 Data Model 38

3.3.2 The Seven Comparison Checks 39

3.3.3 Seccomp Comparison Logic 40

3.4 Capability Normalization Algorithm 41

3.5 Container Runtime Abstraction Layer 42

3.5.1 Docker Runtime Adapter 42

3.5.2 containerd Runtime Adapter 43

3.6 Observability Dashboard 44

3.7 Evaluation Test Case Design 44

3.8 Experimental Environment 45

4 Implementation **46**

4.1 Project Structure 46

4.2 Kubernetes Deployment Manifests 47

4.3 Core Scanning Pipeline 47

4.4 Container Runtime Inspector 48

4.4.1 Docker Runtime Inspector 49

4.4.2 containerd Runtime Inspector 49

4.5 Alert Manager and Output Format 50

4.6 Dashboard Implementation 51

4.7 Deployment Procedure 55

5 Evaluation and Discussion **59**

5.1 Experimental Procedure 59

5.2 Detection Results 60

5.2.1 Layer 1: Specification Violations 60
5.2.2 Layer 2: Drift Findings 60
5.3 True Positive: Privileged Seccomp Stripping 61
5.4 Observability Gap: Docker's Implicit Seccomp Default 62
5.5 Pod 3 vs. Pod 4: The Controlled Comparison 63
5.6 Performance 64
5.7 Discussion 64
5.7.1 Answering the Research Questions 64
5.7.2 Comparison with Existing Tools 65
5.8 Limitations 65
5.8.1 Future Work 66

6 Conclusion **67**
6.1 Summary of Findings 67
6.2 Contributions Restated 68
6.3 Practical Implications 68
6.4 Future Work 68

Bibliography cited **70**

Appendices **75**
Appendix A: Tool Source Code Structure 75
Appendix B: Evaluation Test Pod Specifications 76
Appendix C: Real-Time Dashboard 78
Appendix D: Experimental Environment Details 79

List of Tables

2.1 Security tool landscape 31

3.1 Field mapping between Kubernetes spec and Docker runtime . . . 39
3.2 containerd OCI spec mapping 43
3.3 Evaluation test pod design 44

5.1 Layer 1 specification violations 60
5.2 Layer 2 drift findings 61
5.3 Layer 2 system pod findings 61
5.4 Controlled comparison 63
5.5 Performance characteristics 64
5.6 Tool comparison 65

6.1 Experimental environment 79

List of Figures

1.1 Monolithic architecture. 13
1.2 Microservices architecture. 13
1.3 Multi-layer translation from Kubernetes specification to kernel enforcement. The red-tinted area highlights the enforcement verification gap where security properties can be lost. 14

2.1 Kubernetes security model: from pod securityContext declarations, through Pod Security Standards and admission control, to kernel enforcement via the translation chain. 21
2.2 Static misconfiguration detection workflow. Existing tools analyse Kubernetes manifests before deployment, but none verify that the security context declarations are correctly enforced at runtime – a remaining research gap. 25
2.3 Admission-time enforcement workflow. OPA Gatekeeper and Kyverno verify compliance against the Kubernetes API, but never confirm that the container runtime has actually applied the declared security settings. This assumes faithfulness that may not hold in practice. 26

2.4 Runtime behavioural monitoring tools (Falco, KubeArmor) observe system call activity and detect anomalies, but they do not verify that the container's security context (seccomp, capabilities, etc.) matches the intended enforcement. Their blind spot is the faithfulness of translation from declaration to kernel. 28

2.5 Configuration drift detection tools operate exclusively at the Kubernetes API level, comparing reported state against intended state. They cannot detect runtime-layer drift (e.g., a security property dropped by the container runtime) because the API still returns the intended value. This blind spot leaves security-critical mis-enforcements undiscovered. 30

2.6 Kubernetes security tool landscape: API-level tools (left) and kernel-level monitors (right) operate independently, leaving an enforcement verification gap (red). k8s-psa bridges this gap by directly comparing declared security context against container runtime state. 32

3.1 High-level data flow in k8s-psa. The tool extracts security state from two independent sources, compares them through the drift detector, and outputs findings to a shared volume consumed by the dashboard. 34

3.2 Two-layer scanning pipeline – vertical layout. Left column: Layer 1 detects missing security declarations. Right column: Layer 2 (the novel contribution) identifies runtime drifts. Both streams feed the AlertManager, which produces NDJSON logs for the dashboard. . 35

3.3 Module organisation and data flow of k8s-psa. Nine Python modules are grouped into Data Extraction (blue), Analysis (orange), and Output (green). Arrows show the primary paths of security data from the Kubernetes API through to the dashboard. 36

3.4 Flow of Layer 1 specification-level policy checks. The pod specification is fed into the policy engine, which evaluates all eight configurable security rules. Any violation is emitted as a structured JSON event containing the check details. 37
3.5 Drift detection process (Layer 2). For each container, two independent snapshots are built from the Kubernetes spec and the container runtime. Seven field-by-field comparisons are performed; each mismatch yields a DriftFinding with a severity level. 40
3.6 Seccomp decision logic of the drift detector. The left branch handles the critical case where a declared RuntimeDefault or Localhost profile is missing at runtime. The right branch covers the benign case where an unconfined declaration is overridden by a default runtime profile. 41
3.7 Capability normalization algorithm. The 14 default capabilities form the base set, modified by the pod specification's add/drop lists. 42

4.1 Overall dashboard summary. 52
4.6 Forwarding the dashboard to the host machine 58

5.1 layer1 finding 60
5.2 layer1 finding 60
5.3 layer2 finding 61
5.4 layer2 finding 61
5.5 divergence grapgh 65

Abstract

Container orchestration platforms such as Kubernetes rely on declarative security primitives — seccomp profiles, Linux capabilities, user identity constraints, and namespace isolation — to enforce security boundaries around containerised workloads. A fundamental assumption underlies this model: that the container runtime faithfully translates declared security intent into kernel-level enforcement. This thesis identifies and addresses the *enforcement verification gap*, defined as the absence of tooling that systematically verifies whether declared Kubernetes security contexts are actually enforced by the container runtime.

Existing security tools — OPA Gatekeeper, Kyverno, Kubescape, Trivy Operator — read only the Kubernetes API (what the operator *declared*) and never inspect the container runtime (what the kernel *enforces*). This leaves a dangerous blind spot, particularly for privileged containers: a container running with full host access (all Linux capabilities, unrestricted devices) that also declares a seccomp system-call filter. As we demonstrate, the Docker runtime silently strips that filter, yet no specification-only tool detects the loss. We present k8s-psa (Kubernetes Pod Security Auditor), a DaemonSet-based tool that continuously compares pod security specifications retrieved from the Kubernetes API against the actual runtime state obtained via the container runtime interface. The tool implements a two-layer architecture: Layer 1 performs specification-level policy checks equivalent to existing admission controllers, while Layer 2 — the novel contribution — inspects the container runtime directly and compares it field by field against the declared specification across seven security dimensions.

Through controlled experiments on a Minikube cluster with the Docker runtime, we evaluate the tool across seven purpose-built test scenarios and four Kubernetes control plane components. Our results reveal two categories of findings. First, privileged containers that declare seccomp filtering have that filtering silently stripped by Docker, constituting a genuine enforcement gap invisible to all existing specification-only tools. Second, the Docker runtime metadata interface omits seccomp enforcement status for correctly protected non-privileged containers, constituting an observability gap that produces false positives in naive runtime inspection. These findings demonstrate that the enforcement verification gap is real, that it manifests differently across container configurations, and that runtime inspection must account for the semantic differences between runtime metadata and actual kernel enforcement. The tool, its evaluation framework comprising 44 unit tests and 7 deployment-level test cases, and a real-time observability dashboard are provided as open-source contributions.

Keywords: Kubernetes, container security, seccomp, runtime verification, enforcement drift, DevSecOps, security context, pod security

Chapter 1

Introduction

1.1 Relevance of the Topic (Background and Motivation)

Modern software systems increasingly adopt microservice architectures deployed as containerised workloads on Kubernetes, the industry-standard for container orchestration. As of 2024, over 60% of organisations running containers in production use Kubernetes [1], [2]. This widespread adoption introduces security challenges that traditional perimeter-based approaches cannot adequately address: a larger attack surface distributed across potentially hundreds of containers, complex dependency chains involving third-party images of varying trustworthiness, and highly dynamic runtime behaviour where containers are continuously created, scaled, and terminated [3], [4].

Kubernetes addresses these challenges through declarative security primitives specified in pod security contexts. Administrators express their security intent by declaring constraints. In particular, containers:

- must not run as root,
- must not acquire additional privileges via SUID binaries,

- must use system call filtering (seccomp profiles), and
- must operate with a minimal set of Linux capabilities.

The Kubernetes control plane, container runtime, and underlying Linux kernel are then expected to translate these declarations into actual enforcement.

The ecosystem has developed a mature set of tools for validating these declarations. Admission controllers such as OPA Gatekeeper [5] and Kyverno [6] intercept pod creation requests and reject those that violate organisational security policies. Pod Security Admission (PSA), built into Kubernetes since version 1.25 [7], enforces predefined security profiles (Privileged, Baseline, and Restricted) at the namespace level. Background scanners like Kubescape and Trivy Operator periodically re-evaluate running workloads against security frameworks published by organisations such as the National Security Agency (NSA) and the Center for Internet Security (CIS) [8]. While these tools effectively prevent misconfigured pods from being scheduled, they operate under a critical assumption: that if the Kubernetes API reports a security property, the container runtime has faithfully enforced it. This thesis demonstrates that the assumption does not always hold, particularly for privileged containers.

A **privileged container** is one that runs with $\mathrm{privileged:\ true}$ in its security context. This setting grants the container all Linux capabilities, full access to host devices, and disables seccomp and AppArmor profiles. It essentially provides a root shell on the host, making it a highly dangerous configuration that should be used only when absolutely necessary. Because privileged mode overrides almost all other security settings, any additional restrictions declared alongside it (e.g., seccomp profiles) are likely to be ignored by the runtime, creating a silent but severe security risk.

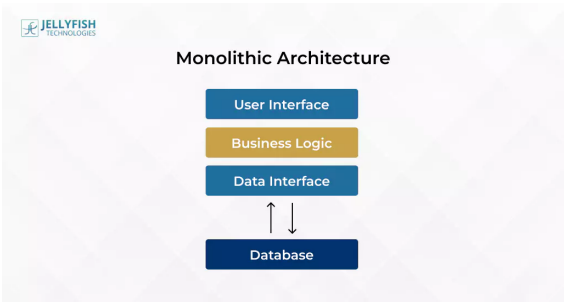


Fig. 1.1. Monolithic architecture.

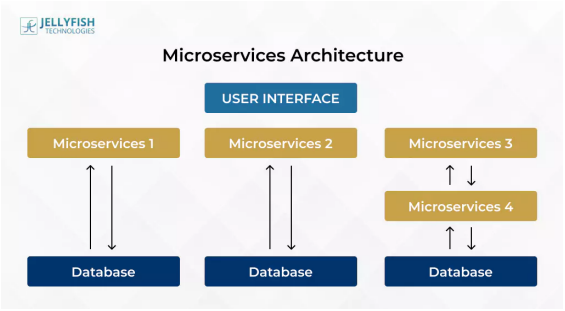


Fig. 1.2. Microservices architecture.

1.2 Problem Statement

Every existing tool in the Kubernetes security ecosystem reads security properties from the Kubernetes API — that is, from the *declared* pod specification. None verify that the container runtime has correctly translated those declarations into kernel-level enforcement. The translation from a Kubernetes pod specification to actual process isolation involves multiple layers, each of which may introduce divergence:

1. The **kubelet** on each node translates the pod specification into a Container Runtime Interface (CRI) request.

2. The **container runtime** (Docker, containerd, or CRI-O) receives the CRI request and generates an Open Container Initiative (OCI) runtime specification.

3. A **low-level runtime** (runc or crun) reads the OCI specification and configures Linux kernel primitives: namespaces, cgroups, capabilities, and seccomp filters.

Each translation step may add defaults, override values, or — critically — silently fail to apply certain security restrictions. The resulting blind spot is what this thesis terms the *enforcement verification gap*: the absence of tooling that systematically verifies whether declared Kubernetes security contexts are faithfully enforced at the container runtime level.

This gap has tangible consequences. An operator who declares seccompProfile.type: RuntimeDefault on a privileged container reasonably expects system call filtering to be active. OPA Gatekeeper reads the Kubernetes API, sees the declaration, and reports compliance. But the Docker runtime silently strips the seccomp filter from privileged containers — and no existing tool detects this.

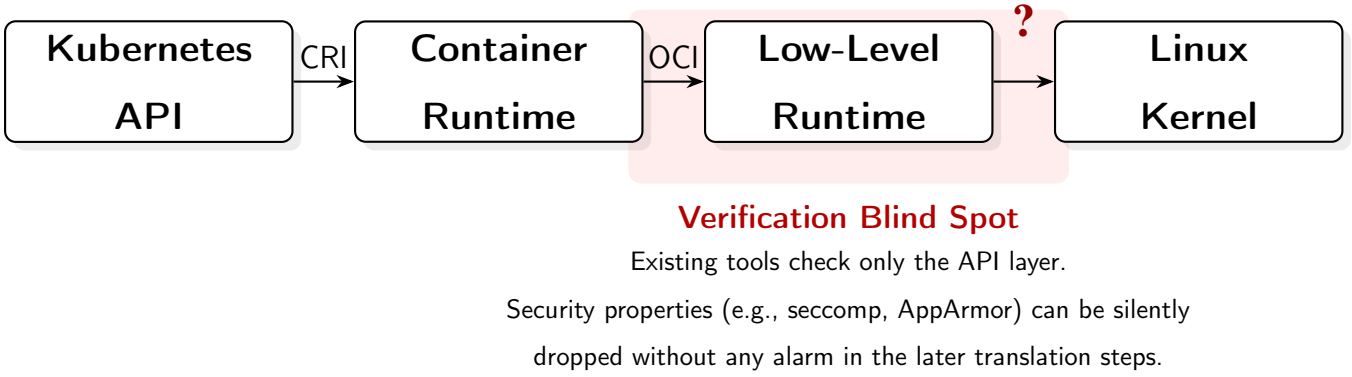


Fig. 1.3. Multi-layer translation from Kubernetes specification to kernel enforcement. The red-tinted area highlights the enforcement verification gap where security properties can be lost.

1.3 Research Questions

This thesis is guided by three research questions:

- **RQ1 (Empirical):** Under what configurations, and to what measurable extent, does the kernel-level security state of running containers diverge from the security context declared in their Kubernetes pod specifications?
- **RQ2 (Engineering feasibility):** Can a node-resident DaemonSet, operating with read-only access to the Kubernetes API and the container runtime socket, detect such divergences with quantifiable accuracy and bounded resource overhead, without modifying the cluster's existing security mechanisms?

- **RQ3:** What categories of divergence occur in practice, and how should each category be classified from a security perspective?

1.4 Aim of the Study

The aim of this thesis is to design, implement, and experimentally evaluate a tool that systematically verifies whether the security context declared in a Kubernetes pod specification is faithfully enforced by the underlying container runtime, and to characterise the categories of divergence that occur in practice between declared and actually enforced security posture.

1.5 Tasks (Objectives) of the Study

To achieve the stated aim, the following tasks were performed:

1. Review the academic and practitioner literature on Kubernetes container security in order to precisely characterise the enforcement verification gap and demonstrate that no existing tool addresses it.

2. Design a two-layer architecture (specification-level checks and runtime drift detection) capable of comparing the declared pod security context against the actual container runtime state across seven security dimensions.

3. Develop a capability normalisation algorithm that accounts for the 14 default Linux capabilities granted by the container runtime, preventing the false-positive findings that would otherwise dominate the output.

4. Implement a runtime abstraction layer that supports both Docker (via cri-dockerd) and containerd (via the CRI interface), so that the tool operates uniformly across heterogeneous clusters.

5. Deploy the tool as a Kubernetes DaemonSet, evaluate it on a controlled set of seven purpose-built test pods and four Kubernetes control plane components, and quantify both its detection capability and its resource overhead.

6. Classify the categories of divergence observed during evaluation and discuss their security implications, including the distinction between genuine enforcement gaps and observability artefacts of the runtime metadata interface.

1.6 Object and Subject of the Study

The object of the study is the security enforcement chain of containerised workloads orchestrated by Kubernetes — comprising the Kubernetes API, the kubelet, the Container Runtime Interface (CRI), the container runtime (Docker, containerd, or CRI-O), the low-level runtime (runc or crun), and the Linux kernel primitives that ultimately enforce security (namespaces, capabilities, seccomp filters, and Linux Security Modules).

The subject of the study is the divergence that may arise between the security posture declared in the Kubernetes pod specification and the security posture actually enforced by the container runtime, and the methods and tooling required to detect such divergence systematically and with bounded resource overhead.

1.7 Scientific Novelty

The scientific novelty of this thesis consists in the following:

1. It is the first work to systematically identify and empirically characterise the *enforcement verification gap* in Kubernetes — the absence of tooling that compares declared security contexts against actual runtime enforcement — and to demonstrate that this gap is exploitable in practice.

2. A novel two-layer verification architecture is proposed in which Layer 1 reproduces the functionality of existing admission controllers, while Layer 2 — the novel contribution — performs field-by-field comparison between the Kubernetes API and the container runtime across seven security dimensions, an approach not implemented by any existing open-source tool.

3. Two previously undocumented phenomena are experimentally demonstrated: (i) Docker silently strips declared seccompProfile filters from privileged containers, producing a true enforcement gap invisible to specification-only tools, and (ii) the Docker inspection interface omits implicit default seccomp profiles from its metadata, producing an observability gap that causes false positives in naïve runtime verification.

4. A capability normalisation algorithm is introduced that correctly reconciles the 14 default Linux capabilities granted by the container runtime with the capabilities.add / capabilities.drop lists declared in the pod specification, eliminating a class of spurious findings that would otherwise overwhelm any field-by-field comparison.

1.8 Contributions

This thesis makes four contributions:

1. **A novel enforcement verification tool** (k8s-psa) that fills the gap identified above by comparing declared Kubernetes security contexts against actual container runtime state across seven security dimensions.

2. **Experimental evidence** of a genuine enforcement gap: privileged containers that declare seccomp profiles have those profiles silently stripped by the Docker runtime.

3. **Discovery of an observability gap** in the Docker runtime metadata interface, where correctly applied implicit seccomp defaults are absent from the `docker inspect` output, producing false positives in naïve runtime inspection.

4. **An open-source evaluation framework** comprising 44 unit tests, 7 deployment-level test cases with documented expected outcomes, and a real-time dashboard for continuous monitoring.

1.9 Thesis Structure

The remainder of this thesis is organised as follows:

- **Chapter 2** reviews the academic and practitioner literature on Kubernetes security, culminating in a precise characterisation of the enforcement verification gap.
- **Chapter 3** presents the design and methodology of k8s-psa, including the two-layer architecture, the capability normalisation algorithm, and the test case design.
- **Chapter 4** describes the implementation, covering the Kubernetes deployment manifests, the runtime inspection backends, and the unit test suite.
- **Chapter 5** presents and discusses the experimental results, including the privileged seccomp stripping finding, the observability gap, and the controlled Pod 3 vs. Pod 4 comparison.
- **Chapter 6** summarises the findings, restates the contributions, and identifies directions for future work.

Chapter 2

Literature Review

This chapter reviews the academic and practitioner literature relevant to Kubernetes container security, with particular attention to the gap between specification-level security tools and runtime enforcement verification. We deliberately walk through each category of tools — static analysis, admission control, runtime monitoring, and drift detection — to show that none combine both the declared specification and the actual runtime state. The chapter concludes by precisely defining the enforcement verification gap that this thesis addresses.

2.1 Kubernetes Security Model

Kubernetes enforces workload security through a layered model centred on the pod specification [9]. The securityContext field defines the intended security posture through several sub-fields:

- privileged – unrestricted host access,
- runAsUser / runAsGroup – process identity,
- capabilities – Linux capability grants and drops,
- seccompProfile – system call filtering,

- readOnlyRootFilesystem – filesystem immutability,
- allowPrivilegeEscalation – controls PR_SET_NO_NEW_PRIVS,
- hostPID, hostNetwork, hostIPC – namespace sharing with the host [10], [11].

Pod Security Standards (PSS), defined by the Kubernetes project, classify pod configurations into three profiles: Privileged (unrestricted), Baseline (prevents known privilege escalations), and Restricted (follows current hardening best practices) [12]. Pod Security Admission (PSA), stable since Kubernetes v1.25 [7], enforces these standards at admission time by intercepting pod creation and update requests.

The translation from pod specification to actual enforcement involves three layers: the kubelet translates the pod spec into a Container Runtime Interface (CRI) request; the container runtime (Docker, containerd, or CRI-O) generates an Open Container Initiative (OCI) runtime specification [13]; and a low-level runtime (runc or crun) configures Linux kernel primitives.

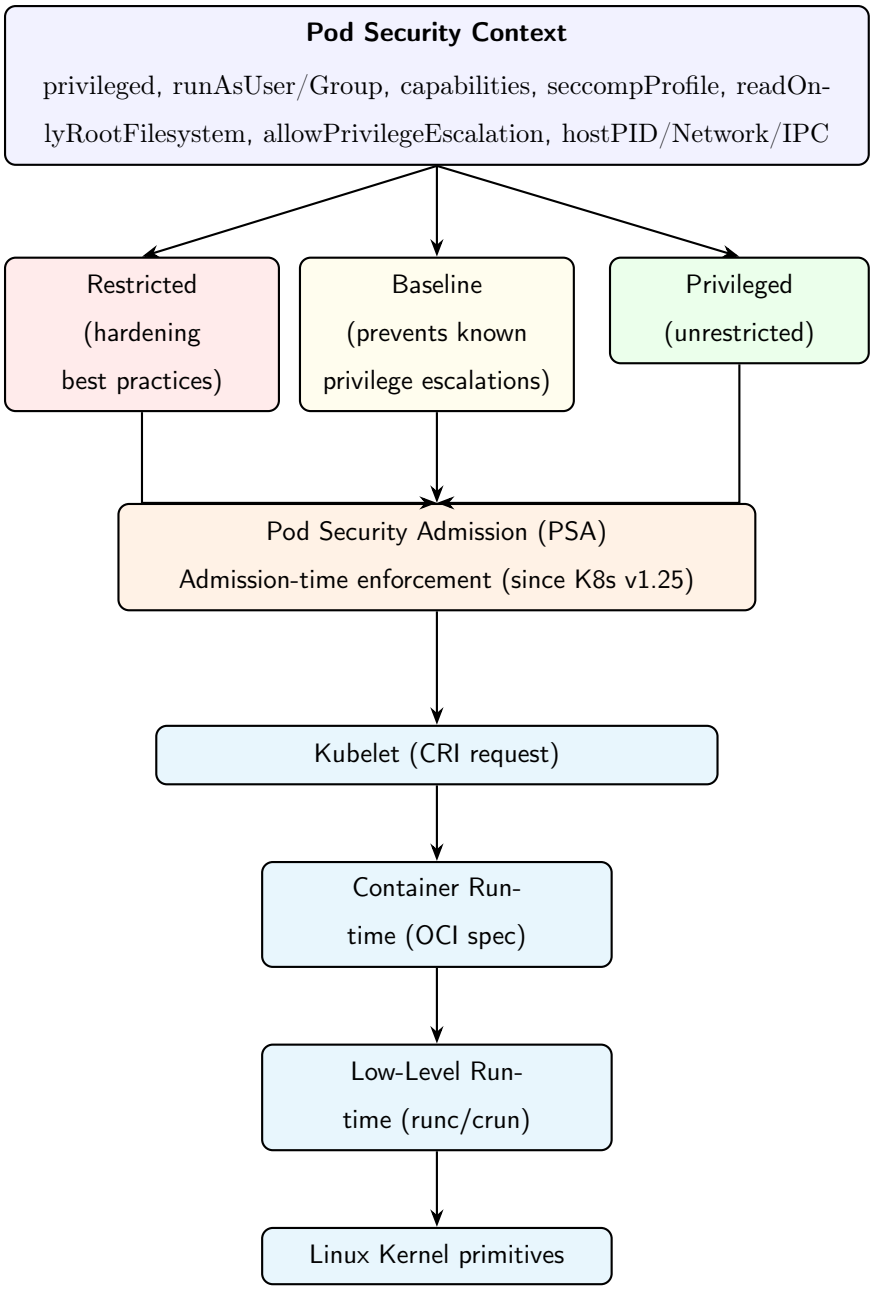


Fig. 2.1. Kubernetes security model: from pod securityContext declarations, through Pod Security Standards and admission control, to kernel enforcement via the translation chain.

2.2 Seccomp Profiles in Kubernetes

Among the security primitives listed in Section 2.1, seccomp deserves a dedicated discussion because it is central to this thesis. *Seccomp* (short for *secure*

computing mode) is a Linux kernel feature that restricts the system calls a process is permitted to make. A seccomp policy is expressed as a Berkeley Packet Filter (BPF) program loaded into the kernel via the seccomp() system call. Once installed, the filter cannot be removed for the lifetime of the process: subsequent system call invocations are evaluated against the BPF program, and disallowed calls return an error or terminate the process depending on the configured action [14], [15].

In Kubernetes, seccomp policies are declared at the pod or container level through the seccompProfile field of the security context. The field accepts three distinct profile types, each with different semantics and operational implications.

2.2.1 The Three Seccomp Profile Types

Unconfined. The container runs with no seccomp filter at all. Every system call is permitted at the kernel level (subject only to capability and namespace restrictions). This is the most permissive option and historically was the default for Kubernetes pods that did not declare a seccomp profile. It exposes the host kernel to the container's full attack surface, including syscalls such as mount, ptrace, kexec_load, and reboot [16].

RuntimeDefault. The container runs with the seccomp profile shipped by the container runtime itself. The Docker default profile is an allow-list expressed as a Berkeley Packet Filter program: it permits the system calls that ordinary application workloads require and denies a curated set of approximately three dozen calls deemed dangerous in a container context [15]. Containerd and CRI-O ship functionally analogous default profiles. The aim is to balance security with compatibility: calls historically associated with container escapes (such as kexec_load, add_key, request_key, and create_module) are blocked, while the syscalls that nearly all workloads invoke are permitted.

Localhost. The container runs with a custom seccomp profile loaded from a JSON file on the node's filesystem, typically stored under /var/lib/kubelet/sec-

comp/. This option enables organisations to define application-specific profiles that are stricter than RuntimeDefault—for instance, a profile that allows only the syscalls a particular static binary actually invokes. The trade-off is operational: Localhost profiles must be distributed to every node before pods that reference them can start, and missing profiles cause pods to fail with a CreateContainerError.

2.2.2 Why Seccomp Matters for This Thesis

Seccomp is the security primitive most affected by the enforcement verification gap. Three properties make it particularly susceptible to silent failure:

- **Asymmetric default behaviour.** Unlike capabilities (which the kernel grants by default), seccomp is opt-in: a pod without a seccompProfile declaration runs unconfined unless the kubelet has been explicitly configured otherwise. The Kubernetes Enhancement Proposal KEP-2413 [17] began the work of changing this default, but the migration is not yet complete across the ecosystem.

- **Privileged-mode override.** As documented in the Moby project issue tracker [18], Docker silently strips declared seccomp profiles from privileged containers. The pod specification still reports seccompProfile.type: RuntimeDefault, but the kernel applies no filter. This is the central enforcement gap that this thesis quantifies.

- **Opaque metadata.** The Docker inspection interface does not record the implicit application of RuntimeDefault: when cri-dockerd forwards a RuntimeDefault declaration to Docker, no explicit SecurityOpt entry is added, and Docker's built-in default is applied invisibly. As a result, docker inspect cannot distinguish a correctly protected container from a genuinely unprotected one—an observability gap that complicates runtime verification (discussed in detail in Chapter 5).

These three properties together explain why specification-level tools that read only the Kubernetes API cannot detect seccomp drift, and why runtime metadata alone is insufficient. Definitive verification requires reading the kernel's own seccomp enforcement status, which is exposed through /proc/<pid>/status as a Seccomp field whose value indicates the active mode (0 = disabled, 1 = strict, 2 = BPF filter mode).

2.3 Static Misconfiguration Detection

A significant body of work focuses on detecting security misconfigurations in Kubernetes manifests before deployment.

Rahman et al. [19] studied misconfigurations in Kubernetes deployment files at scale, examining over two thousand manifests sourced from 92 public repositories. Their analysis uncovered recurring security oversights such as pods running without any security context, containers that omit resource boundaries, and workloads deployed into the default namespace. The authors noted explicitly that their methodology was limited to static manifest analysis and that verifying whether the declared configurations are correctly enforced at runtime remained an open research direction.

Malul et al. [20] proposed GenKubeSec, a system that fine-tunes large language models for detecting, localising, and remediating Kubernetes misconfigurations in YAML files. GenKubeSec achieves high accuracy on static manifests but operates entirely at the pre-deployment stage. It should be noted that this work remains an unreviewed preprint at the time of writing.

Sgan Cohen et al. [21] presented KubeGuard, which combines configuration files with runtime logs to assess Kubernetes security. KubeGuard focuses primarily on Role-Based Access Control (RBAC) and NetworkPolicy analysis rather than securityContext enforcement verification. This work also remains an unreviewed preprint.

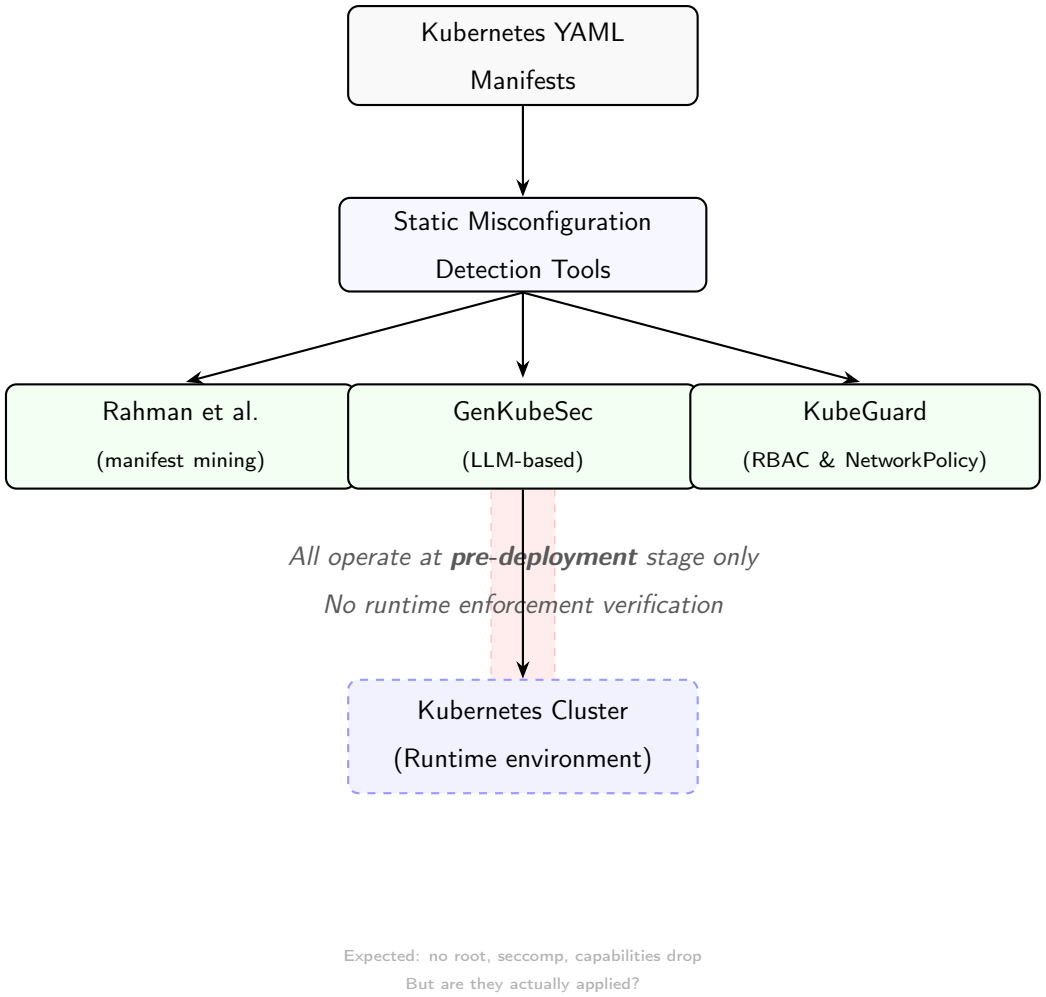


Fig. 2.2. Static misconfiguration detection workflow. Existing tools analyse Kubernetes manifests before deployment, but none verify that the security context declarations are correctly enforced at runtime – a remaining research gap.

2.4 Admission-Time Policy Enforcement

OPA Gatekeeper [5] brings the Open Policy Agent framework into the Kubernetes ecosystem by encoding security constraints as CustomResourceDefinitions that a validating admission webhook evaluates at pod creation time. Gatekeeper also offers a periodic audit capability that re-checks existing cluster resources, but this audit operates solely against the Kubernetes API; the container runtime is never queried.

Kyverno [6] takes a Kubernetes-native approach to policy management,

allowing operators to express validation, mutation, and generation rules using standard YAML manifest syntax rather than the Rego language that Gatekeeper requires. Its background scan controller periodically rechecks running workloads. As with Gatekeeper, however, Kyverno reads exclusively from the Kubernetes API.

Both tools are highly effective at preventing non-compliant pods from being admitted to the cluster. However, they operate on a fundamental assumption: that if the Kubernetes API reports a security property, the container runtime has faithfully enforced it. This assumption, as this thesis demonstrates, does not always hold [3].

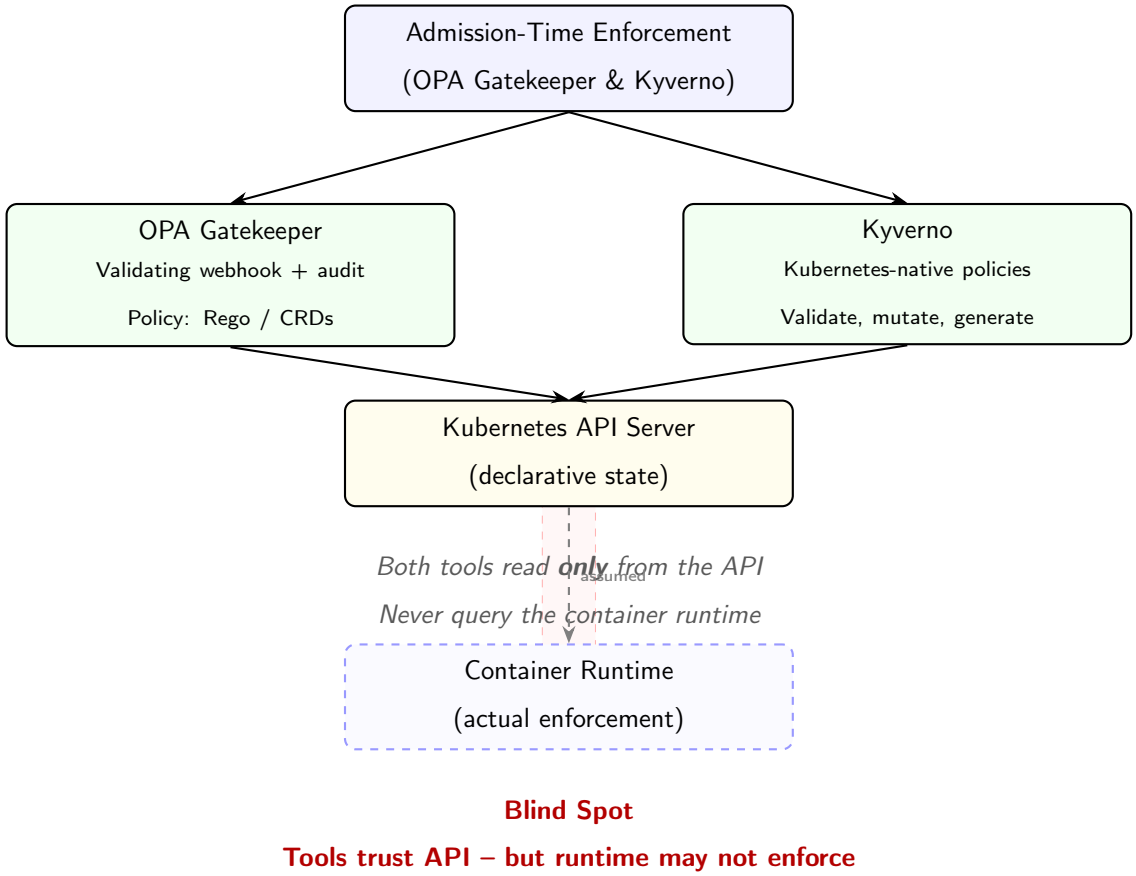


Fig. 2.3. Admission-time enforcement workflow. OPA Gatekeeper and Kyverno verify compliance against the Kubernetes API, but never confirm that the container runtime has actually applied the declared security settings. This assumes faithfulness that may not hold in practice.

2.5 Runtime Behavioural Monitoring

A separate class of tools monitors actual container behaviour at the kernel level. Falco, which graduated within the CNCF project hierarchy, attaches eBPF probes to kernel functions and uses a rule engine to flag anomalous events — unexpected shell processes, connections to suspicious endpoints, or attempts at privilege escalation [22], [23]. KubeArmor offers a comparable detection layer and can also enforce containment policies through Linux Security Modules.

These tools address a complementary problem: they monitor what containers *do* at the system call level. However, they do not verify whether the security configuration that constrains those actions matches what was declared in the pod specification. Falco can detect that a container called a dangerous system call, but it cannot determine whether a seccomp profile was supposed to block that call and failed to do so.

Morić, Dakić, and Čavala [24] evaluated the security posture of Kubernetes control plane components against CIS benchmarks, finding that default installations of Kubernetes leave several hardening measures unapplied. Their work highlights the prevalence of security gaps in production-like environments but focuses on static CIS benchmark compliance rather than runtime enforcement verification.

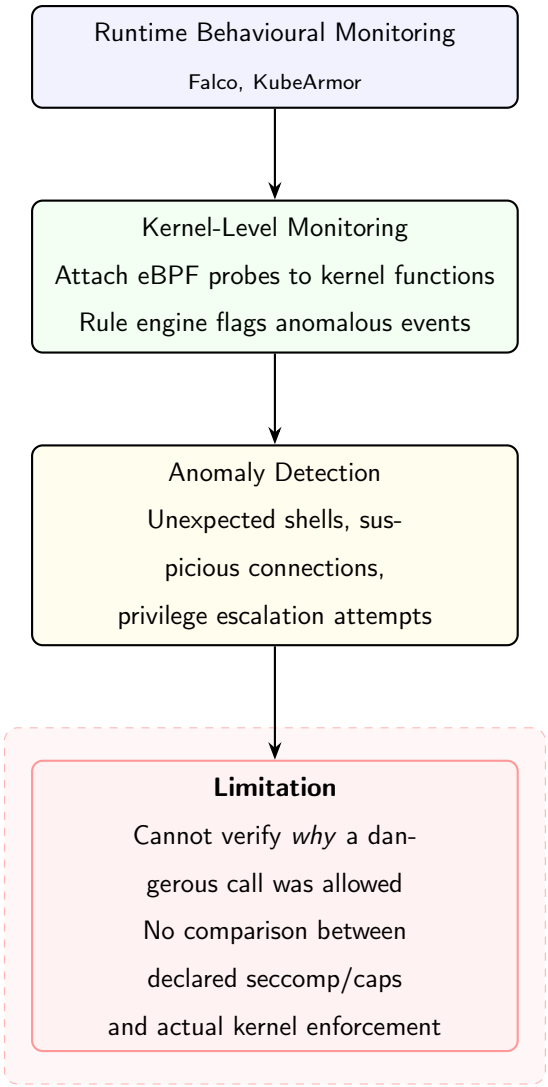


Fig. 2.4. Runtime behavioural monitoring tools (Falco, KubeArmor) observe system call activity and detect anomalies, but they do not verify that the container's security context (seccomp, capabilities, etc.) matches the intended enforcement. Their blind spot is the faithfulness of translation from declaration to kernel.

2.6 Configuration Drift Detection

Configuration drift — the divergence between the intended state and the actual state of a system — has been studied in several contexts within the Kubernetes ecosystem.

Kubediff [25] compares the running state of Kubernetes resources against version-controlled manifests stored in git, alerting when differences are detected. GitOps tools such as ArgoCD and FluxCD implement continuous reconciliation loops that detect and optionally remediate manifest-level drift. The Kubernetes project blog [26] described using admission controllers to detect runtime container drift caused by `kubectl exec` commands that modify container state after deployment. Komodor [27] documented approaches for detecting Kubernetes configuration drift across deployments.

All of these tools detect drift at the *Kubernetes API level* — they compare what the API currently reports against what was intended. None extend below the API to the container runtime layer. A container that has been modified via `kubectl exec` can be detected by image hash comparison, but a container whose seccomp profile was silently stripped by the runtime cannot be detected by any of these tools, because the Kubernetes API still reports the intended seccomp configuration.

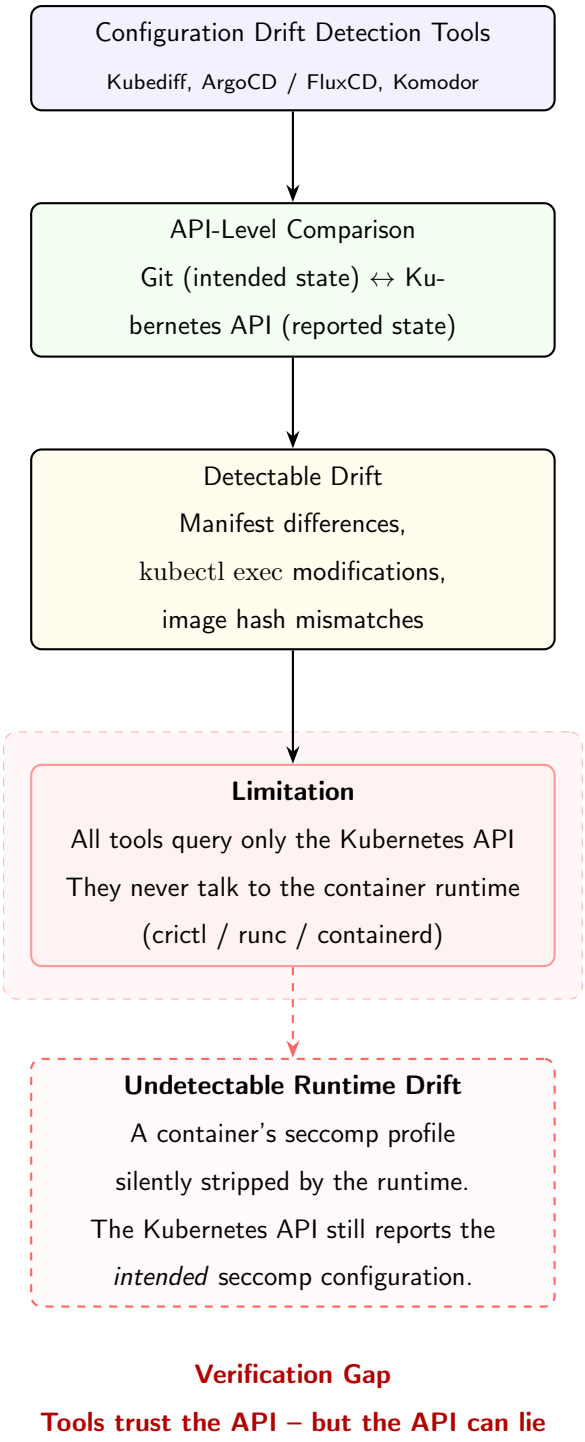


Fig. 2.5. Configuration drift detection tools operate exclusively at the Kubernetes API level, comparing reported state against intended state. They cannot detect runtime-layer drift (e.g., a security property dropped by the container runtime) because the API still returns the intended value. This blind spot leaves security-critical mis-enforcements undiscovered.

2.7 The Identified Gap

The literature reveals a clear gap between specification-level tools and kernel-level monitors. Table 2.1 summarises the existing tool landscape and the position of this thesis. As shown, every tool operates either on the Kubernetes API (left side) or on kernel events (right side), but none bridge the two.

Table 2.1. Kubernetes security tool landscape and the enforcement verification gap.

Tool	**What It Checks**	**Data Source**	**Detects Runtime Drift?**
OPA Gatekeeper	Spec vs. policy	K8s API	No
Kyverno	Spec vs. policy	K8s API	No
Kubescape	Spec vs. frameworks	K8s API	No
Trivy Operator	Spec + image vulns	K8s API + registry	No
Falco	Syscall behaviour	Kernel (eBPF)	No*
Kubediff	Spec vs. git	K8s API + git	No
kube-bench	Node config vs. CIS	Node filesystem	No
k8s-psa (this thesis)	**Spec vs. runtime**	**K8s API + container runtime**	**Yes**

*Falco detects syscall anomalies but does not compare runtime state against the declared specification.

Several academic works have directly motivated the need for runtime enforcement verification. Kermabon-Bobinnec et al. [28] demonstrated in their ProSPEC system that temporal inconsistency between orchestration and runtime layers can be exploited to bypass security policies in containerised environments. Shamim et al. [29] evaluated dynamic

application security testing tools against CIS benchmarks for Kubernetes deployments and concluded that future work should investigate methods for detecting configuration violations dynamically at runtime. Wong et al. [1] and Devi Priya et al. [30] both identified runtime configuration drift detection as an under-explored area in their respective comprehensive surveys of container security.

CloudSecBurrito [31] articulated the gap concisely from a practitioner perspective: Kubernetes can ensure the container runtime *receives* the security intent, but it cannot confirm the kernel actually *enforced* it.

No existing open-source tool systematically compares Kubernetes API security declarations against container runtime state [32], [33]. This thesis fills that gap with k8s-psa, whose design we present next.

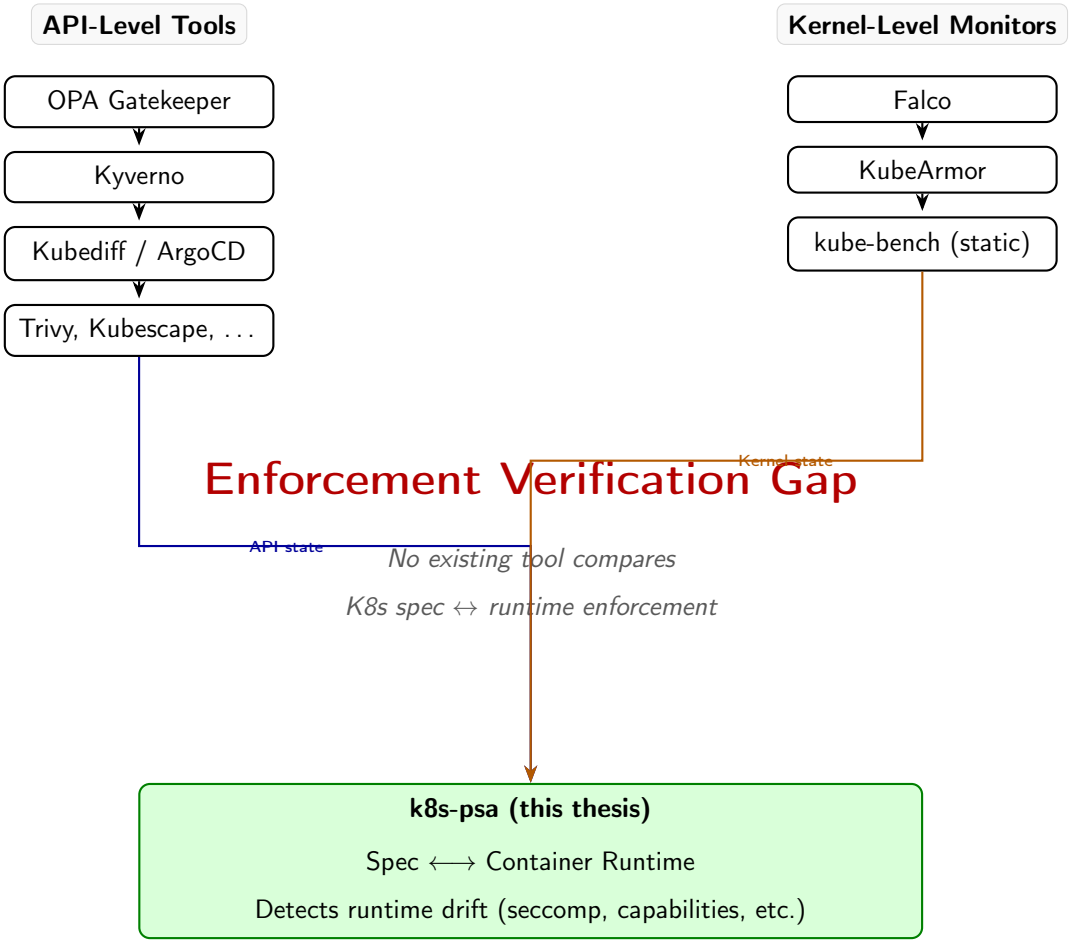


Fig. 2.6. Kubernetes security tool landscape: API-level tools (left) and kernel-level monitors (right) operate independently, leaving an enforcement verification gap (red). k8s-psa bridges this gap by directly comparing declared security context against container runtime state.

Chapter 3

Design and Methodology

Having established that no existing tool compares declared specification with actual runtime state, this chapter presents the design of k8s-psa — a tool that does exactly that. We first describe the overall architecture, then detail the two scanning layers, the capability normalisation algorithm, the runtime back-end auto-detection, the observability dashboard, and the evaluation test case design.

3.1 Architectural Overview

The fundamental design principle of k8s-psa is that security posture verification requires comparing two independent data sources: the *declared* security intent stored in the Kubernetes API and the *actual* security enforcement at the container runtime level. No single data source is sufficient. The Kubernetes API reflects what the operator *intended*; the container runtime state reflects what the kernel *enforces*.

k8s-psa is deployed as a Kubernetes DaemonSet [34], ensuring that exactly one scanner pod runs on every node in the cluster. This deployment model provides three properties: complete node coverage (every container on every node is inspected), co-location with the container runtime socket (the scanner pod mounts the Docker or containerd socket from the host), and automatic scaling (when nodes are added or removed, the DaemonSet controller adjusts accordingly).

Each scanner pod executes a periodic scan cycle consisting of two sequential phases, referred to as Layer 1 and Layer 2. The scan interval is configurable, with a default of 60 seconds. Figure 3.1 illustrates the high-level data flow.

In addition, Figure 3.2 breaks down the internal scanning pipeline, showing how Layer 1 and Layer 2 work together.

The tool consists of nine Python modules organised into three functional groups. Figure 3.3 shows the grouping and the primary data flows between the modules.

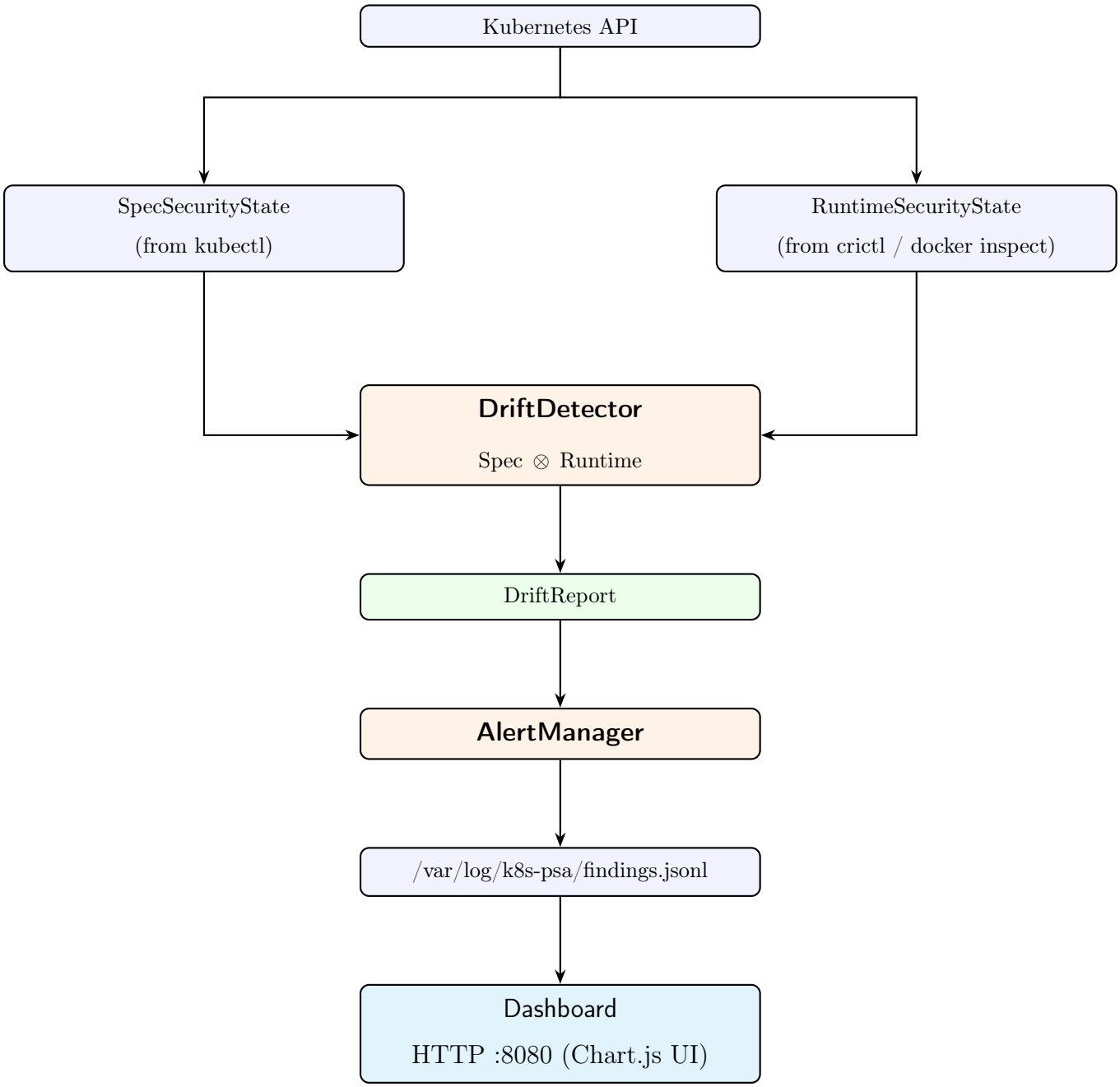


Fig. 3.1. High-level data flow in k8s-psa. The tool extracts security state from two independent sources, compares them through the drift detector, and outputs findings to a shared volume consumed by the dashboard.

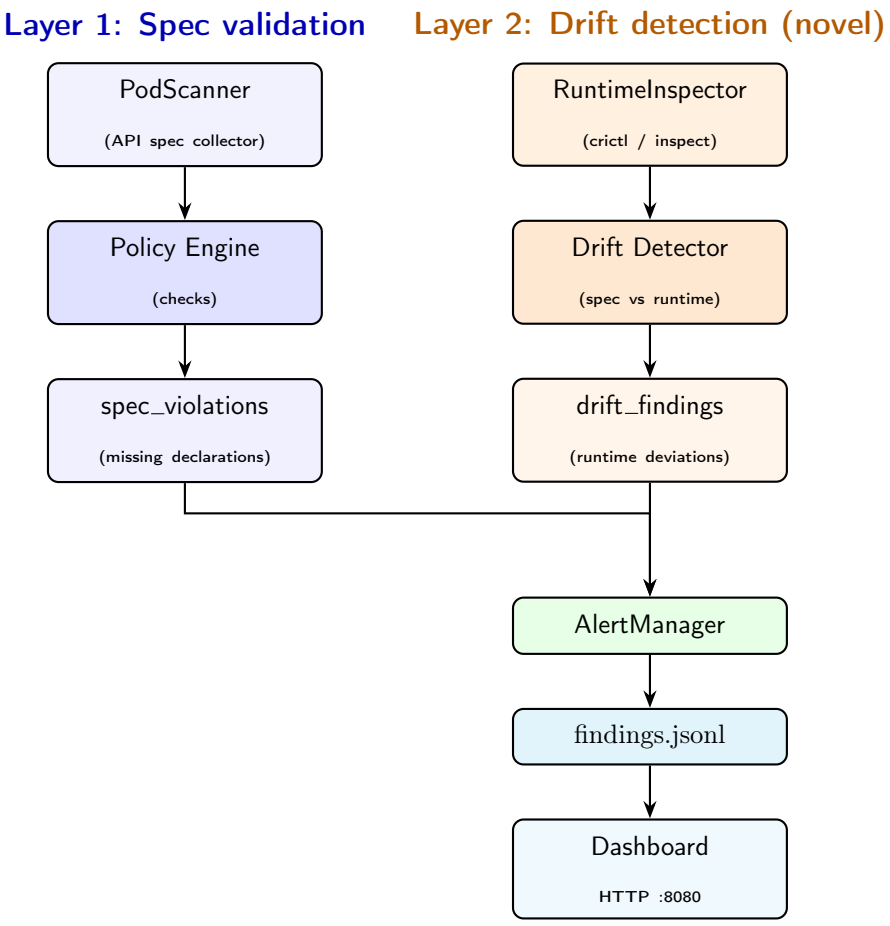


Fig. 3.2. Two-layer scanning pipeline – vertical layout. Left column: Layer 1 detects missing security declarations. Right column: Layer 2 (the novel contribution) identifies runtime drifts. Both streams feed the AlertManager, which produces NDJSON logs for the dashboard.

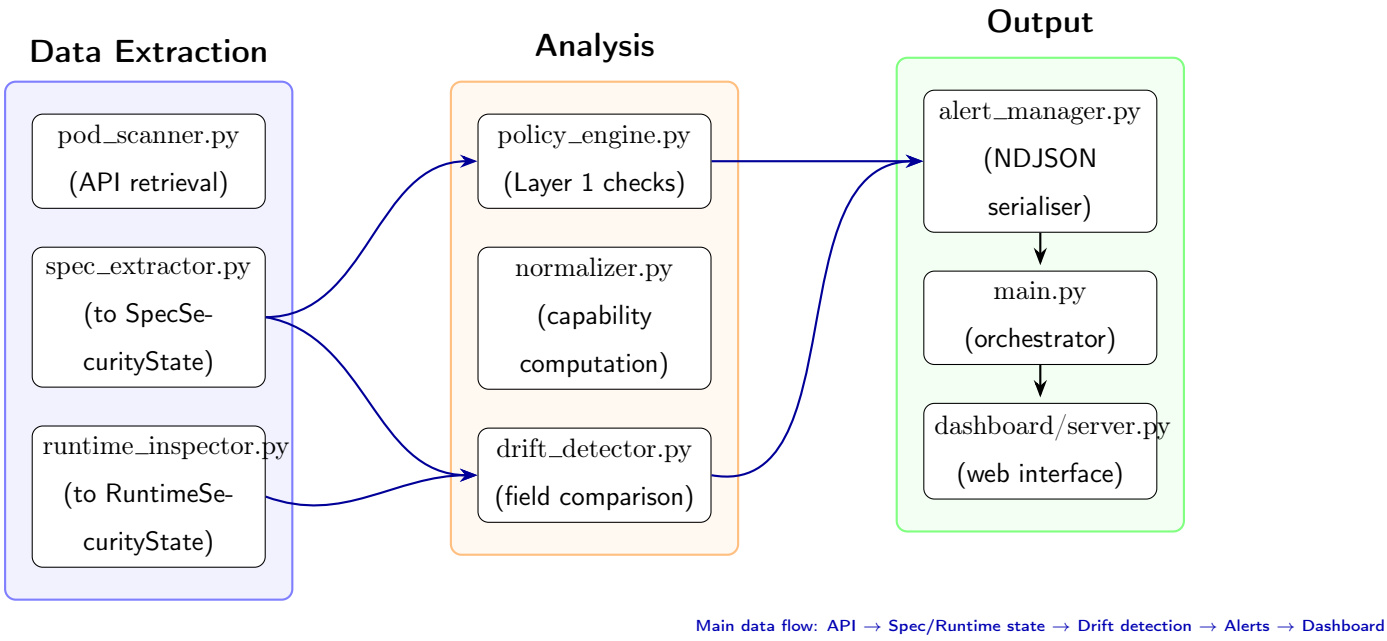


Fig. 3.3. Module organisation and data flow of k8s-psa. Nine Python modules are grouped into Data Extraction (blue), Analysis (orange), and Output (green). Arrows show the primary paths of security data from the Kubernetes API through to the dashboard.

3.2 Layer 1: Specification-Level Policy Checks

Layer 1 reads pod specifications from the Kubernetes API and evaluates them against eight configurable security policies defined in policies/default.yaml. This layer operates entirely on declared specifications, providing functionality equivalent to existing tools such as OPA Gatekeeper [5] and Kyverno [6]. Its purpose within k8s-psa is twofold: to provide a baseline against which Layer 2 findings can be contextualised, and to detect pods that lack security declarations entirely (which Layer 2 cannot flag, since the absence of a declaration means there is no intent to verify).

The eight policy checks are:

1. **Privileged mode** (high): Flags containers with privileged: true.
2. **Root user** (high): Flags containers with explicit runAsUser: 0 or missing runAsNonRoot: true.
3. **Host PID** (medium): Flags pods with hostPID: true.
4. **Host network** (medium): Flags pods with hostNetwork: true.
5. **Host IPC** (medium): Flags pods with hostIPC: true.
6. **Seccomp profile** (medium): Flags pods without a seccomp profile declaration.
7. **Dangerous capabilities** (medium): Flags containers that add capabilities listed in a configurable deny list (default: SYS_ADMIN, NET_ADMIN, SYS_PTRACE).

8. **Read-only root filesystem** (low): Flags containers without readOnlyRootFilesystem: true.

Each violation is emitted as a structured JSON event containing the check identifier, severity, pod name, namespace, and a human-readable message.

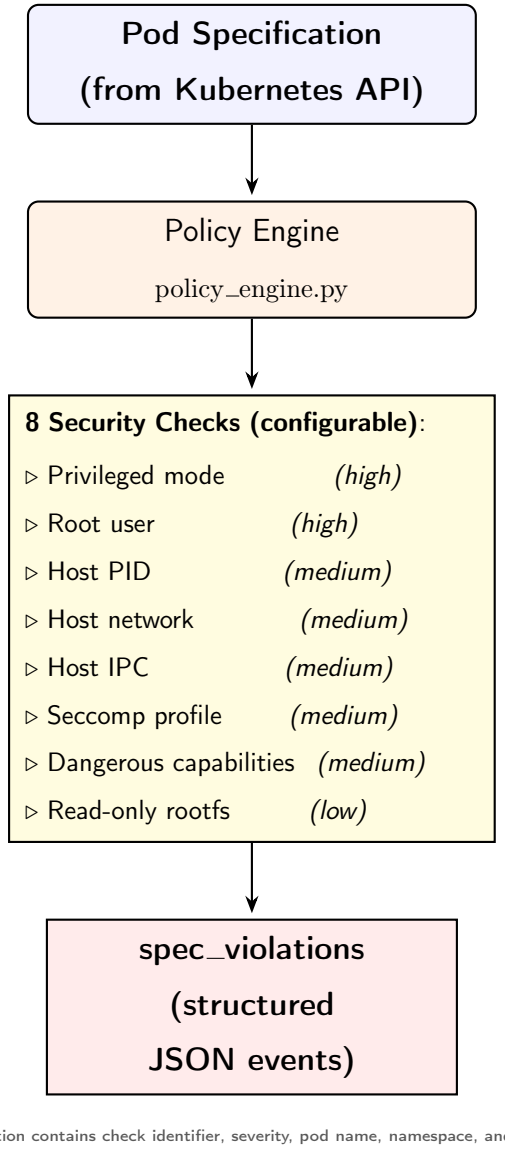


Fig. 3.4. Flow of Layer 1 specification-level policy checks. The pod specification is fed into the policy engine, which evaluates all eight configurable security rules. Any violation is emitted as a structured JSON event containing the check details.

3.3 Layer 2: Runtime Drift Detection

Layer 2 is the novel contribution of this work and the direct answer to the enforcement verification gap. For each running container, it constructs two data structures — SpecSecurityState and RuntimeSecurityState — and compares them field by field across seven security dimensions. This comparison is the first time a tool has systematically verified that the security context declared in the Kubernetes API is actually present in the running

container.

3.3.1 Data Model

The SpecSecurityState dataclass captures the security-relevant fields from the Kubernetes pod specification:

Listing 3.1. SpecSecurityState data model (simplified)

```
@dataclass
class SpecSecurityState:
    container_name: str
    pod_name: str
    namespace: str
    privileged: bool = False
    run_as_user: Optional[int] = None
    run_as_group: Optional[int] = None
    read_only_root_filesystem: bool = False
    allow_privilege_escalation: Optional[bool] = None
    capabilities_add: List[str] = field(default_factory=list)
    capabilities_drop: List[str] = field(default_factory=list)
    seccomp_profile_type: Optional[str] = None
    host_network: bool = False
    host_pid: bool = False
    host_ipc: bool = False
```

The RuntimeSecurityState dataclass captures the corresponding fields from the container runtime:

Listing 3.2. RuntimeSecurityState data model (simplified)

```
@dataclass
class RuntimeSecurityState:
    container_id: str
    container_name: str
    uid: int = 0
    gid: int = 0
    capabilities_effective: List[str] = field(default_factory=list)
    capabilities_bounding: List[str] = field(default_factory=list)
    seccomp_profile: Optional[dict] = None
```

```
root_readonly: bool = False
no_new_privileges: bool = False
masked_paths: List[str] = field(default_factory=list)
namespace_pid: int = 0
namespace_network: int = 0
namespace_ipc: int = 0
```

3.3.2 The Seven Comparison Checks

The drift detector implements seven comparison methods, each responsible for one security dimension. Table 3.1 summarises the mapping between Kubernetes spec fields and Docker runtime fields.

Table 3.1. Field mapping between Kubernetes securityContext and Docker inspect output

Security Dimension	Kubernetes Spec Path	Docker Inspect Path
Privileged mode	securityContext .privileged	HostConfig.Privileged + inferred from all capabilities, no seccomp, no masked paths
User identity	securityContext. runAsUser / runAsGroup	Config.User
Capabilities	securityContext. capabilities .add / .drop	HostConfig.CapAdd / CapDrop
Seccomp profile	securityContext. seccompProfile .type	HostConfig.SecurityOpt
Read-only rootfs	securityContext .readOnlyRootFilesystem	HostConfig.ReadonlyRootfs
Privilege escalation	securityContext .allowPrivilegeEscalation	HostConfig.SecurityOpt (no-new-privileges flag)
Host namespaces	hostPID / hostNetwork / hostIPC	HostConfig.PidMode / NetworkMode / IpcMode

Each comparison produces zero or more DriftFinding objects classified into three severity levels:

- **CRITICAL**: A security restriction was declared in the specification but is absent at runtime. The container is less secure than intended. Example: seccompProfile.type: RuntimeDefault is declared, but no seccomp filter is active.
- **WARNING**: A capability or privilege is present at runtime that was not explicitly requested in the specification. Example: CAP_SYS_ADMIN is active at runtime but was not listed in capabilities.add.

- **INFO**: The runtime is more restrictive than the specification requires, or a known benign default was applied. Example: the runtime applies a seccomp profile even though none was declared.

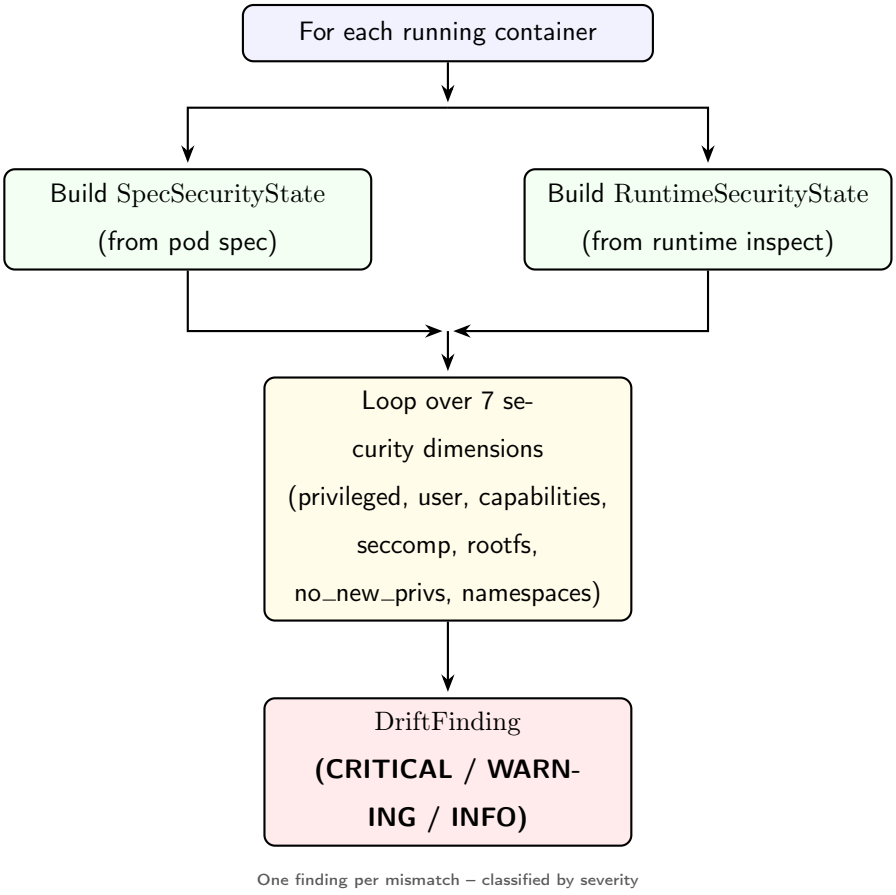


Fig. 3.5. Drift detection process (Layer 2). For each container, two independent snapshots are built from the Kubernetes spec and the container runtime. Seven field-by-field comparisons are performed; each mismatch yields a DriftFinding with a severity level.

3.3.3 Seccomp Comparison Logic

The seccomp check deserves special attention because it is at the centre of the primary research finding. The comparison logic is:

1. If the specification declares RuntimeDefault or Localhost AND the runtime has no seccomp profile: emit CRITICAL (declared restriction absent).
2. If the specification declares Unconfined or is empty AND the runtime has a seccomp profile: emit INFO (runtime is stricter than declared).
3. Otherwise: no finding (spec and runtime agree).

This logic correctly detects the case where Docker strips seccomp from privileged containers (case 1), but as discussed in Chapter 5, it also fires for non-privileged containers where Docker applies its implicit default profile without recording it in the inspection metadata.

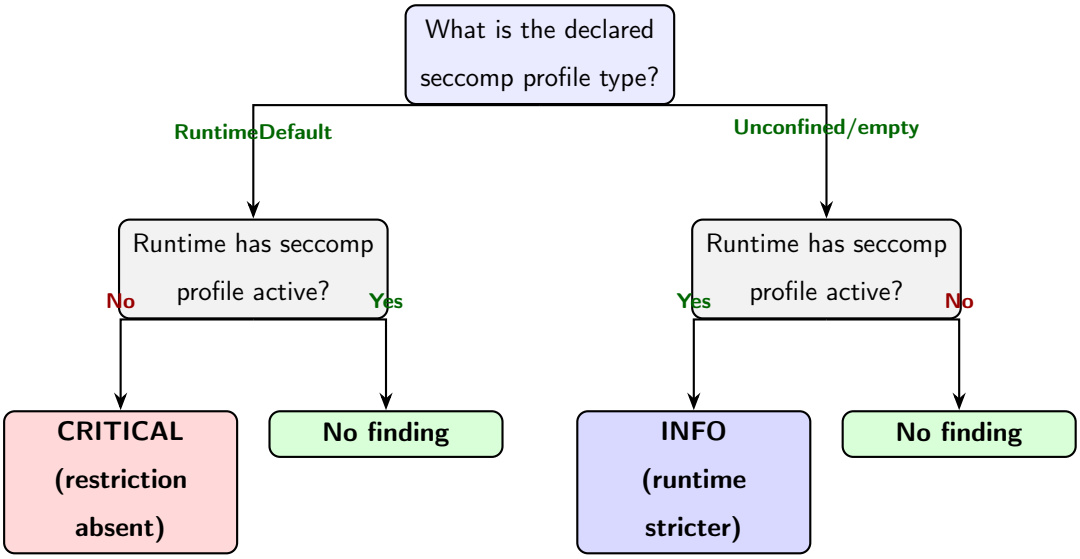


Fig. 3.6. Seccomp decision logic of the drift detector. The left branch handles the critical case where a declared RuntimeDefault or Localhost profile is missing at runtime. The right branch covers the benign case where an unconfined declaration is overridden by a default runtime profile.

3.4 Capability Normalization Algorithm

The Linux kernel grants 14 default capabilities to non-privileged containers [14], [16]. When a Kubernetes pod specification omits the capabilities field entirely, the runtime will still grant these 14 defaults. Comparing the raw specification (which lists zero capabilities) against the raw runtime state (which lists 14 capabilities) would produce 14 false-positive WARNING findings.

To prevent this, the normalizer computes the *expected* capability set from the specification using Algorithm 3.7.

The 14 default capabilities are: CAP_CHOWN, CAP_DAC_OVERRIDE, CAP_FSETID, CAP_FOWNER, CAP_MKNOD, CAP_NET_RAW, CAP_SETGID, CAP_SETUID, CAP_SETFCAP, CAP_SETPCAP, CAP_NET_BIND CAP_SYS_CHROOT, CAP_KILL, and CAP_AUDIT_WRITE.

When privileged: true is declared, the Docker runtime grants all 41 Linux capabilities. The normalizer does not expand the expected set to 41 in this case, because the specification does not *request* those additional capabilities explicitly. This design decision means that privileged containers always produce WARNING findings for the 27 non-default capabilities — correctly flagging the capability inflation that accompanies privileged mode.

Algorithm 1: ComputeExpectedCapabilities(*add*, *drop*)

Input: *add* = list of capabilities to add; *drop* = list of capabilities to drop

Output: *expected* = set of capabilities the runtime should grant

if "ALL" $\in$ *drop* **then**
    expected $\leftarrow \emptyset$
else
    expected $\leftarrow$ DefaultCapabilities $\setminus$ *drop*
end if
expected $\leftarrow$ *expected* $\cup$ *add*
return *expected*

Fig. 3.7. Capability normalization algorithm. The 14 default capabilities form the base set, modified by the pod specification's add/drop lists.

3.5 Container Runtime Abstraction Layer

Kubernetes clusters do not converge on a single container runtime. Production environments use Docker (via cri-dockerd), containerd, and CRI-O interchangeably, each exposing its own inspection interface, metadata schema, and command-line tooling. A runtime-coupled drift detector would therefore be unable to operate across heterogeneous clusters. To address this, k8s-psa introduces a Container Runtime Abstraction Layer that hides the runtime-specific details behind a uniform interface and produces a single $\mathrm{RuntimeSecurityState}$ representation regardless of the underlying technology.

The abstraction layer performs automatic adapter selection at startup using the following decision procedure:

1. Check for the Docker socket at $/\mathrm{var}/\mathrm{run}/\mathrm{docker.sock}$. If present, instantiate the Docker runtime adapter.
2. Otherwise, check for the containerd socket at $/\mathrm{var}/\mathrm{run}/\mathrm{containerd}/\mathrm{containerd.sock}$. If present, instantiate the containerd runtime adapter (via the crictl interface).
3. If neither socket is found, log an error and skip Layer 2 inspection for the current scan cycle.

3.5.1 Docker Runtime Adapter

The Docker runtime adapter invokes $\mathrm{docker\ ps\ {-}{-}format\ json}$ to enumerate running containers, filters out infrastructure pod sandboxes by excluding entries whose $\mathrm{io.kubernetes.container.name}$ label equals POD, and then executes $\mathrm{docker\ inspect}$ <id> for each remaining container. From the resulting JSON, the adapter extracts HostConfig.Privileged, HostConfig.CapAdd, HostConfig.CapDrop, HostConfig.SecurityOpt, HostCon-

fig.ReadonlyRootfs, Config.User, HostConfig.PidMode, HostConfig.NetworkMode, and HostConfig.IpcMode, and converts them into the canonical RuntimeSecurityState fields.

3.5.2 containerd Runtime Adapter

The containerd runtime adapter uses the CRI command-line client (crictl) rather than a Docker-style inspection API, because containerd is a CRI-native runtime and does not expose the Docker-compatible HostConfig structure. Inspection proceeds in three stages:

1. **Enumeration.** The command crictl ps -o json lists all running containers along with their Kubernetes pod labels, namespaces, and container IDs.
2. **Per-container inspection.** For each container, crictl inspect <id> returns a JSON document containing two top-level sections. The status section provides the container's identity and labels; the info.runtimeSpec section embeds the complete OCI runtime specification that was used to create the container.
3. **OCI runtime specification parsing.** The adapter extracts security-relevant fields directly from the OCI spec rather than from runtime-specific metadata. The mapping is shown in Table 3.2.

Table 3.2. Mapping from canonical RuntimeSecurityState fields to OCI runtime specification paths (containerd runtime adapter)

Canonical field	**OCI runtime spec path** (info.runtimeSpec)
Privileged (effective)	Inferred from full capability set, absent seccomp, and absent maskedPaths
Capabilities (effective)	process.capabilities.effective
Capabilities (bounding)	process.capabilities.bounding
User UID / GID	process.user.uid / process.user.gid
Seccomp profile	linux.seccomp (full BPF rule set, including defaultAction and per-syscall rules)
Read-only root filesystem	root.readonly
noNewPrivileges flag	process.noNewPrivileges
Namespace modes	linux.namespaces[] (per-namespace type and path)
Masked paths	linux.maskedPaths

The containerd runtime adapter offers an important advantage over the Docker runtime adapter with respect to seccomp visibility. The OCI runtime specification embedded in info.runtimeSpec contains the complete seccomp profile, including defaultAction (typically SCMP_ACT_ERRNO for the runtime default) and the full list of allowed syscalls. This eliminates the observability gap discussed in Chapter 5: where the Docker runtime adapter can only confirm that *some* seccomp policy is active by the presence of an entry in HostConfig.SecurityOpt, the containerd runtime adapter can verify the actual BPF rules that the kernel is enforcing.

Both adapters emit RuntimeSecurityState objects with identical field semantics, which preserves the runtime-agnosticity of the drift detection logic. The capability normalisation algorithm described in Section 3.4, the field-by-field comparison logic described in Section 3.3.2, and the severity classification scheme apply unchanged regardless of which adapter produced the runtime state.

3.6 Observability Dashboard

The real-time dashboard is deployed as a separate Kubernetes Deployment that mounts the same hostPath volume as the scanner. It consists of a zero-dependency Python HTTP server (dashboard/server.py) that serves an HTML page with embedded Chart.js visualisations.

The dashboard provides six visualisation panels: (1) drift findings by severity (doughnut chart), (2) drift findings by security field (horizontal bar chart), (3) drift findings by category (horizontal bar chart), (4) specification violations by check (horizontal bar chart), (5) findings per scan cycle (time-series line chart), and (6) scrollable tables listing all unique findings sorted by severity.

The frontend polls the /api/stats endpoint every 10 seconds. The server reads the NDJSON log files, aggregates findings by severity, field, category, and scan cycle, and returns a JSON statistics object. This architecture requires no external database and operates within the resource constraints of a Minikube single-node cluster.

3.7 Evaluation Test Case Design

To evaluate the tool systematically, seven test pods were designed to cover the complete spectrum of Kubernetes security configurations. Each pod isolates one or more security variables while holding others constant. Table 3.3 summarises the test case design.

Table 3.3. Evaluation test pods and their security configurations

Pod	Name	Configuration and Purpose
1	Fully Hardened	runAsUser:1000, drop:[ALL], readOnlyRootFilesystem:true, allowPrivilegeEscalation:false, seccomp:RuntimeDefault. False-positive control.
2	No Security Context	No securityContext at all. Tests that absent declarations produce no drift.
3	Privileged + Seccomp	privileged:true with seccomp:RuntimeDefault. Key scenario: Docker strips seccomp from privileged containers.
4	Privileged, No Seccomp	privileged:true without seccomp declaration. Scientific control for Pod 3.

Pod	Name	Configuration and Purpose
5	Partial Security	drop:[NET_RAW, SYS_ADMIN], seccomp:RuntimeDefault, runAsUser:1000. Tests partial capability drops.
6	No Privilege Escalation	allowPrivilegeEscalation:false with full hardening. Tests noNewPrivileges mapping.
7	Host Namespaces	hostPID:true, hostNetwork:true, hostIPC:true. Tests namespace mode translation.

The critical comparison is between Pods 3 and 4. Both configure privileged: true, resulting in identical runtime states (all 41 capabilities, no seccomp, no masked paths). The only difference is the seccomp declaration in Pod 3's specification. If the drift detector correctly identifies Pod 3 as having a CRITICAL seccomp mismatch and Pod 4 as having no seccomp mismatch, it demonstrates that the tool correctly distinguishes between "restriction declared but not enforced" and "restriction never declared."

Additionally, four Kubernetes control plane components — etcd, kube-apiserver, kube-controller-manager, and kube-scheduler — serve as real-world test subjects with elevated privileges and known security configurations defined by the Kubernetes project itself.

3.8 Experimental Environment

All experiments were conducted on a single-node Minikube cluster with the following configuration:

- **Host OS:** Ubuntu 22.04 LTS
- **Minikube version:** v1.35.0 with Docker driver
- **Kubernetes version:** v1.32.0
- **Container runtime:** Docker Engine 27.4.1 (accessed via cri-dockerd)
- **Scanner interval:** 60 seconds
- **Test image:** busybox:latest with sleep 3600 entrypoint
- **Data collection:** 3 complete scan cycles per experiment

The choice of Minikube with Docker driver is deliberate. While production clusters predominantly use containerd, the Docker driver enables direct access to docker inspect output, making it possible to study the full translation chain from Kubernetes spec to Docker runtime state. The limitation of this choice is discussed in Chapter 5.

Chapter 4

Implementation

This chapter translates the design of Chapter 3 into a concrete implementation. We describe the Kubernetes deployment manifests, the core scanning pipeline, the runtime inspection back-ends, the alert output system, and the real-time dashboard. All source code is written in Python 3.12 and packaged as two Docker container images.

4.1 Project Structure

The final implementation comprises 37 files organised as follows:

Listing 4.1. Project directory structure

```
k8s-psa/
  src/
    models.py            # Data structures
    normalizer.py        # Capability normalization
    pod_scanner.py       # Kubernetes API retrieval
    spec_extractor.py    # Pod spec parsing
    runtime_inspector.py# Docker/crictl inspection
    policy_engine.py     # Layer 1 checks
    drift_detector.py    # Layer 2 checks (7 comparisons)
    alert_manager.py     # NDJSON output
    main.py              # Orchestrator
  dashboard/
    server.py            # HTTP server + Chart.js UI
    Dockerfile
  tests/
```

```
  test_all.py          # 34 core unit tests
  test_evaluation.py   # 10 evaluation unit tests
  fixtures/            # JSON test fixtures
  evaluation/          # 7 test pod YAMLs
deployments/
  all-in-one.yaml      # Namespace, RBAC, DaemonSet, Dashboard
policies/default.yaml # Configurable policy thresholds
Dockerfile             # Scanner image
deploy.sh              # One-command deployment
```

4.2 Kubernetes Deployment Manifests

The deployments/all-in-one.yaml manifest defines five Kubernetes resources:

1. **Namespace** (k8s-psa-system): Isolates the tool's resources from user workloads.
2. **ClusterRole and ClusterRoleBinding**: Grants the scanner pod read-only access to the Kubernetes API for pod listing across all namespaces.
3. **DaemonSet** (k8s-psa): Deploys the scanner on every node. The pod mounts three volumes: the Docker socket (/var/run/docker.sock) as read-only for runtime inspection, the containerd socket (/var/run/containerd/containerd.sock as read-only for the alternative backend, and a hostPath volume (/var/log/k8s-psa/) for writing findings in NDJSON format.
4. **Deployment** (k8s-psa-dashboard): Runs a single replica of the dashboard server, mounting the same hostPath volume as read-only.
5. **Service** (k8s-psa-dashboard): Exposes the dashboard on port 8080, accessible via kubectl port-forward.

The scanner container image is built from a minimal Python base with kubectl, docker CLI, and crictl installed. Resource limits are set to 100m CPU and 128 MiB memory.

4.3 Core Scanning Pipeline

The main orchestrator (main.py) implements the following scan cycle:

Listing 4.2. Main scan cycle (simplified)

```
def scan_node(node_name, interval=60):
    scanner = PodScanner()
    inspector = RuntimeInspector()  # auto-detects backend
```

```
policy = PolicyEngine("policies/default.yaml")
detector = DriftDetector()
alerts = AlertManager("/var/log/k8s-psa/")

while True:
    pods = scanner.get_pods(node_name)

    # Layer 1: spec-level policy checks
    for pod in pods:
        violations = policy.check(pod)
        for v in violations:
            alerts.emit_spec_violation(v)

    # Layer 2: drift detection
    containers = inspector.get_all_containers()
    for cid, runtime_state in containers.items():
        spec_state = extract_spec(pods, runtime_state)
        if spec_state:
            report = detector.detect(spec_state, runtime_state)
            for finding in report.findings:
                alerts.emit_drift_finding(finding)

    alerts.emit_scan_summary(len(pods), ...)
    time.sleep(interval)
```

The PodScanner executes kubectl get pods –all-namespaces -o json and filters pods assigned to the current node using the spec.nodeName field. The SpecExtractor then parses each pod's JSON, merging pod-level security context fields (such as hostNetwork, hostPID, seccompProfile) with container-level fields (such as privileged, capabilities, runAsUser).

4.4 Container Runtime Inspector

The Container Runtime Inspector is the implementation of the runtime abstraction layer described in Section 3.5. It comprises two concrete inspectors that share a common interface and produce identical RuntimeSecurityState objects: one for clusters that use Docker (via cri-dockerd) and one for clusters that use containerd directly through the CRI interface. The Docker inspector was used for the evaluation reported in Chapter 5; the containerd inspector

was implemented and unit-tested but not exercised in cluster experiments because the Minikube test environment used the Docker driver.

4.4.1 Docker Runtime Inspector

The Docker runtime inspector is the primary implementation used in our experimental evaluation. It performs the following steps for each container:

1. Execute docker ps --format '{{.ID}}\t{{.Names}}\t{{.Labels}}' to enumerate running containers.
2. Filter containers by the presence of the io.kubernetes.container.name label, excluding infrastructure POD containers.
3. For each Kubernetes container, execute docker inspect <container_id> and parse the JSON output.
4. Extract security-relevant fields from the HostConfig and Config sections.

The seccomp profile detection parses the SecurityOpt array, searching for entries matching the pattern seccomp=<value>. If no seccomp entry is found, the seccomp_profile field is set to None, which the drift detector interprets as "no seccomp filter active." As discussed in Chapter 5, this interpretation is correct for privileged containers but produces false positives for non-privileged containers where Docker applies its implicit default.

The capability extraction combines HostConfig.CapAdd (explicitly added capabilities) with the effective capability set inferred from the container's privileged status. For privileged containers, all 41 Linux capabilities are assumed effective.

The namespace mode detection maps Docker's string-valued PidMode, NetworkMode, and IpcMode to integer codes: "host" $\rightarrow$ 2 (host namespace), anything else $\rightarrow$ 0 (isolated namespace).

4.4.2 containerd Runtime Inspector

The containerd runtime inspector targets clusters that interact with the container runtime directly through the Container Runtime Interface, without an intermediate Docker daemon. Its implementation differs from the Docker inspector in three respects.

First, container enumeration uses the CRI command-line client. The inspector executes crictl ps -o json, which returns a JSON array of running containers along with their Kubernetes labels (pod name, namespace, container name) attached during pod creation. The same filtering rule applies: entries with io.kubernetes.container.name == POD are excluded because they represent the pod-level pause containers rather than the application workloads under inspection.

Second, per-container inspection uses crictl inspect <container_id>, which returns a richer JSON document than docker inspect. The response has two top-level sections of interest: a status section that mirrors the labels returned by ps, and an info.runtimeSpec section that embeds the complete OCI runtime specification used to create the container. The inspector parses the OCI spec directly rather than relying on derived metadata.

Third, the extraction logic targets OCI spec paths rather than Docker-style HostConfig fields. Capabilities are read from process.capabilities.effective and process.capabilities.bounding, which are explicit arrays of capability names. The seccomp filter is read from linux.seccomp, which contains the complete BPF rule set including

defaultAction and the full list of permitted syscalls. Read-only root filesystem state comes from root.readonly, the noNewPrivileges flag from process.noNewPrivileges, namespace modes from linux.namespaces[], and masked paths from linux.maskedPaths.

Because the OCI spec records all of these values explicitly, the containerd inspector does not suffer the seccomp observability gap that affects the Docker inspector. Where docker inspect reports an empty SecurityOpt for both "no seccomp" and "implicit default seccomp" cases, crictl inspect either records the complete profile under linux.seccomp or returns no linux.seccomp field at all—an unambiguous signal.

Both inspectors construct RuntimeSecurityState objects with identical field semantics. As a result, the drift detector, normaliser, and alert manager operate without modification regardless of which inspector produced the runtime state.

4.5 Alert Manager and Output Format

The AlertManager writes findings as newline-delimited JSON (NDJSON) to two files on the shared hostPath volume:

- findings.jsonl: Contains both spec_violation and drift_finding events.
- scans.jsonl: Contains one summary record per scan cycle with aggregate counts.

Each drift finding record contains the following fields:

Listing 4.3. Example NDJSON drift finding record

```
{
  "type": "drift_finding",
  "timestamp": "2026-04-13T18:45:30.503918+00:00",
  "pod": "test-3-privileged-seccomp",
  "namespace": "default",
  "container": "priv-seccomp",
  "container_id": "a1b2c3d4e5f6",
  "field": "seccompProfile",
  "severity": "critical",
  "category": "seccomp_mismatch",
  "description": "Spec declares seccomp type 'RuntimeDefault'
    but the runtime has NO seccomp filter (unconfined)",
  "declared": "RuntimeDefault",
  "actual": "unconfined (null)"
}
```

This format enables both real-time dashboard consumption and offline analysis with standard JSON processing tools (e.g., jq, Python, or the provided analyze.py script).

4.6 Dashboard Implementation

The dashboard is a single-file Python HTTP server that serves an HTML page with embedded Chart.js visualisations. The server implements two endpoints:

- GET /: Serves the HTML page with Chart.js loaded from a CDN.
- GET /api/stats: Reads the NDJSON log files, aggregates findings, and returns a JSON statistics object.

The HTML page uses JavaScript setInterval to poll /api/stats every 10 seconds and update six Chart.js instances. The dashboard requires no external database, no npm dependencies, and no build step. It is packaged as a separate Docker image containing only Python and the server script.

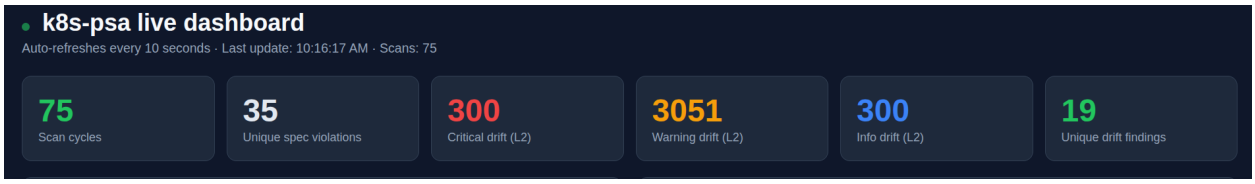
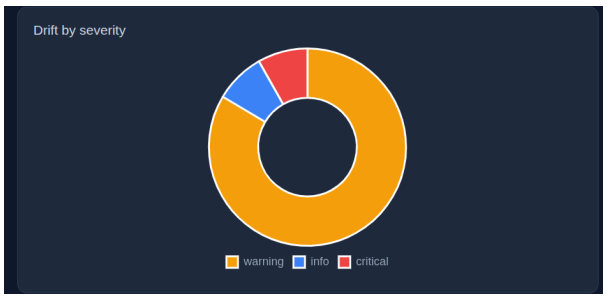


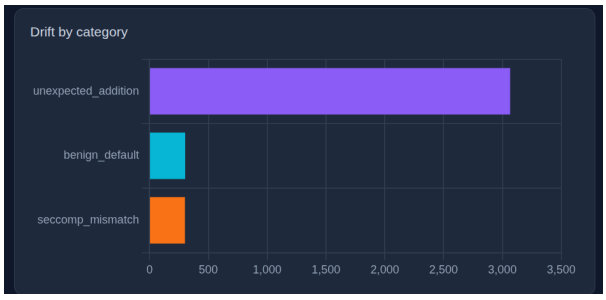
Fig. 4.1. Overall dashboard summary.



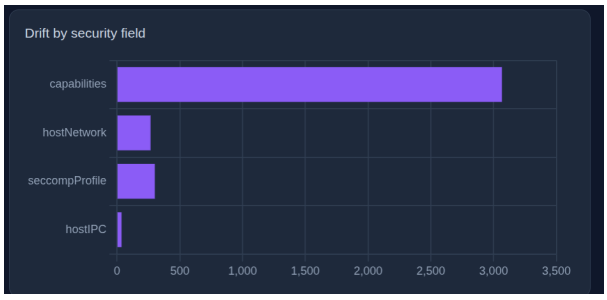
(a) Drift by severity



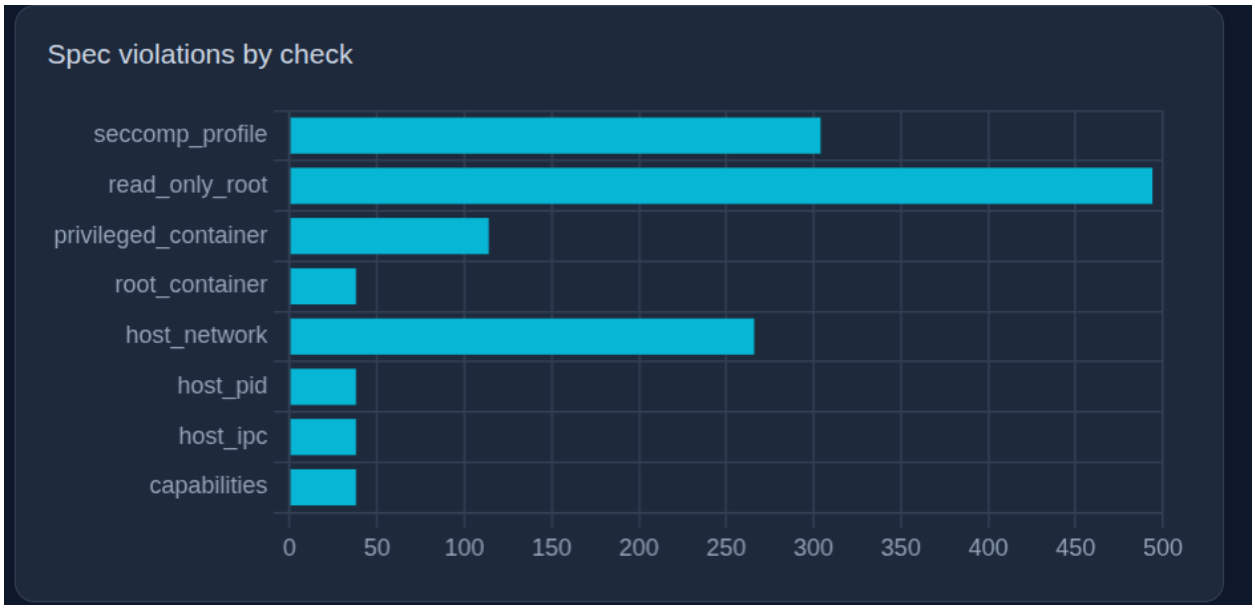
(b) Findings per scan cycle



(c) Drift by category

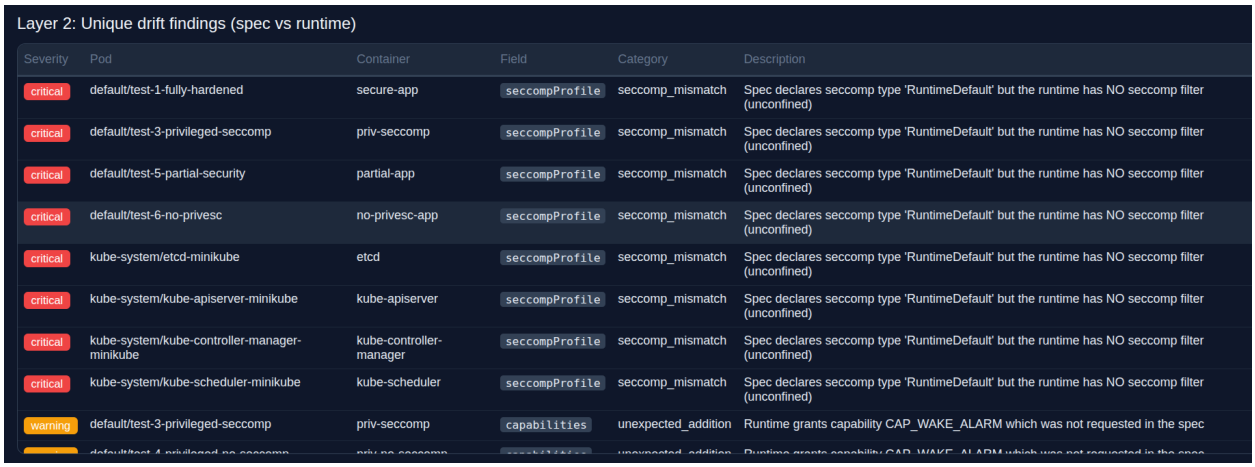


(d) Drift by security field



(e) Spec violations by check

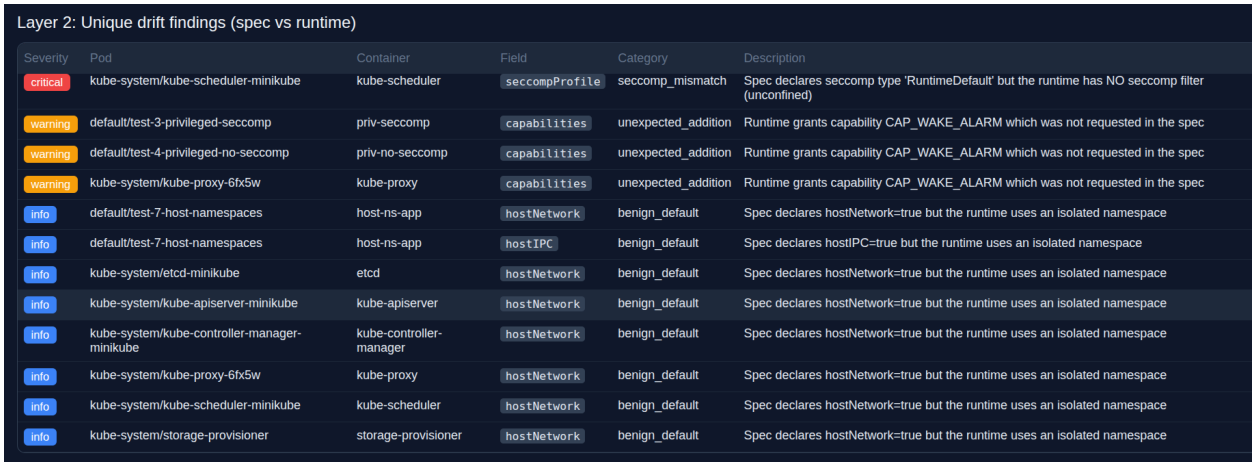
(f) Complete set of PSA dashboard views – summary and bar charts.



Layer 2: Unique drift findings (spec vs runtime)

Severity	Pod	Container	Field	Category	Description
critical	default/test-1-fully-hardened	secure-app	seccompProfile	seccomp_mismatch	Spec declares seccomp type 'RuntimeDefault' but the runtime has NO seccomp filter (unconfined)
critical	default/test-3-privileged-seccomp	priv-seccomp	seccompProfile	seccomp_mismatch	Spec declares seccomp type 'RuntimeDefault' but the runtime has NO seccomp filter (unconfined)
critical	default/test-5-partial-security	partial-app	seccompProfile	seccomp_mismatch	Spec declares seccomp type 'RuntimeDefault' but the runtime has NO seccomp filter (unconfined)
critical	default/test-6-no-privesc	no-privesc-app	seccompProfile	seccomp_mismatch	Spec declares seccomp type 'RuntimeDefault' but the runtime has NO seccomp filter (unconfined)
critical	kube-system/etcd-minikube	etcd	seccompProfile	seccomp_mismatch	Spec declares seccomp type 'RuntimeDefault' but the runtime has NO seccomp filter (unconfined)
critical	kube-system/kube-apiserver-minikube	kube-apiserver	seccompProfile	seccomp_mismatch	Spec declares seccomp type 'RuntimeDefault' but the runtime has NO seccomp filter (unconfined)
critical	kube-system/kube-controller-manager-minikube	kube-controller-manager	seccompProfile	seccomp_mismatch	Spec declares seccomp type 'RuntimeDefault' but the runtime has NO seccomp filter (unconfined)
critical	kube-system/kube-scheduler-minikube	kube-scheduler	seccompProfile	seccomp_mismatch	Spec declares seccomp type 'RuntimeDefault' but the runtime has NO seccomp filter (unconfined)
warning	default/test-3-privileged-seccomp	priv-seccomp	capabilities	unexpected_addition	Runtime grants capability CAP_WAKE_ALARM which was not requested in the spec

(a) Layer 2 findings (part 1)



Layer 2: Unique drift findings (spec vs runtime)

Severity	Pod	Container	Field	Category	Description
critical	kube-system/kube-scheduler-minikube	kube-scheduler	seccompProfile	seccomp_mismatch	Spec declares seccomp type 'RuntimeDefault' but the runtime has NO seccomp filter (unconfined)
warning	default/test-3-privileged-seccomp	priv-seccomp	capabilities	unexpected_addition	Runtime grants capability CAP_WAKE_ALARM which was not requested in the spec
warning	default/test-4-privileged-no-seccomp	priv-no-seccomp	capabilities	unexpected_addition	Runtime grants capability CAP_WAKE_ALARM which was not requested in the spec
warning	kube-system/kube-proxy-6fx5w	kube-proxy	capabilities	unexpected_addition	Runtime grants capability CAP_WAKE_ALARM which was not requested in the spec
info	default/test-7-host-namespaces	host-ns-app	hostNetwork	benign_default	Spec declares hostNetwork=true but the runtime uses an isolated namespace
info	default/test-7-host-namespaces	host-ns-app	hostIPC	benign_default	Spec declares hostIPC=true but the runtime uses an isolated namespace
info	kube-system/etcd-minikube	etcd	hostNetwork	benign_default	Spec declares hostNetwork=true but the runtime uses an isolated namespace
info	kube-system/kube-apiserver-minikube	kube-apiserver	hostNetwork	benign_default	Spec declares hostNetwork=true but the runtime uses an isolated namespace
info	kube-system/kube-controller-manager-minikube	kube-controller-manager	hostNetwork	benign_default	Spec declares hostNetwork=true but the runtime uses an isolated namespace
info	kube-system/kube-proxy-6fx5w	kube-proxy	hostNetwork	benign_default	Spec declares hostNetwork=true but the runtime uses an isolated namespace
info	kube-system/kube-scheduler-minikube	kube-scheduler	hostNetwork	benign_default	Spec declares hostNetwork=true but the runtime uses an isolated namespace
info	kube-system/storage-provisioner	storage-provisioner	hostNetwork	benign_default	Spec declares hostNetwork=true but the runtime uses an isolated namespace

(b) Layer 2 findings (part 2)

(c) Layer 2: Unique drift findings (spec vs runtime).

Layer 1: Unique spec violations

Severity	Pod	Check	Message
low	k8s-psa-system/k8s-psa-dashboard-6648b4c6c9-rxh6g	read_only_root	Container dashboard root filesystem is not read-only
medium	k8s-psa-system/k8s-psa-tf4xs	seccomp_profile	No seccomp profile or unsafe profile:
low	k8s-psa-system/k8s-psa-tf4xs	read_only_root	Container k8s-psa root filesystem is not read-only
medium	kube-system/coredns-668d6bf9bc-48m7s	seccomp_profile	No seccomp profile or unsafe profile:
medium	kube-system/etcd-minikube	host_network	Pod uses host network namespace
low	kube-system/etcd-minikube	read_only_root	Container etcd root filesystem is not read-only
medium	kube-system/kube-apiserver-minikube	host_network	Pod uses host network namespace
low	kube-system/kube-apiserver-minikube	read_only_root	Container kube-apiserver root filesystem is not read-only
medium	kube-system/kube-controller-manager-minikube	host_network	Pod uses host network namespace
low	kube-system/kube-controller-manager-minikube	read_only_root	Container kube-controller-manager root filesystem is not read-only
high	kube-system/kube-proxy-6fx5w	privileged_container	Container kube-proxy is running in privileged mode
medium	kube-system/kube-proxy-6fx5w	host_network	Pod uses host network namespace

(a) Spec violations (page 1)

Layer 1: Unique spec violations

Severity	Pod	Check	Message
medium	kube-system/kube-scheduler-minikube	host_network	Pod uses host network namespace
low	kube-system/kube-scheduler-minikube	read_only_root	Container kube-scheduler root filesystem is not read-only
medium	kube-system/storage-provisioner	host_network	Pod uses host network namespace
medium	kube-system/storage-provisioner	seccomp_profile	No seccomp profile or unsafe profile:
low	kube-system/storage-provisioner	read_only_root	Container storage-provisioner root filesystem is not read-only
medium	default/test-2-no-security	seccomp_profile	No seccomp profile or unsafe profile:
low	default/test-2-no-security	read_only_root	Container bare-app root filesystem is not read-only
high	default/test-3-privileged-seccomp	privileged_container	Container priv-seccomp is running in privileged mode
low	default/test-3-privileged-seccomp	read_only_root	Container priv-seccomp root filesystem is not read-only
high	default/test-4-privileged-no-seccomp	privileged_container	Container priv-no-seccomp is running in privileged mode
high	default/test-4-privileged-no-seccomp	root_container	Container priv-no-seccomp runs as UID 0 (root)
medium	default/test-4-privileged-no-seccomp	seccomp_profile	No seccomp profile or unsafe profile:

(b) Spec violations (page 2)

(c) Layer 1: Unique spec violations (1/2).

Layer 1: Unique spec violations

Severity	Pod	Check	Message
low	default/test-3-privileged-seccomp	read_only_root	Container priv-seccomp root filesystem is not read-only
high	default/test-4-privileged-no-seccomp	privileged_container	Container priv-no-seccomp is running in privileged mode
high	default/test-4-privileged-no-seccomp	root_container	Container priv-no-seccomp runs as UID 0 (root)
medium	default/test-4-privileged-no-seccomp	seccomp_profile	No seccomp profile or unsafe profile:
low	default/test-4-privileged-no-seccomp	read_only_root	Container priv-no-seccomp root filesystem is not read-only
low	default/test-5-partial-security	read_only_root	Container partial-app root filesystem is not read-only
medium	default/test-7-host-namespaces	host_network	Pod uses host network namespace
medium	default/test-7-host-namespaces	host_pid	Pod uses host PID namespace
medium	default/test-7-host-namespaces	host_ipc	Pod uses host IPC namespace
medium	default/test-7-host-namespaces	seccomp_profile	No seccomp profile or unsafe profile:
medium	default/test-7-host-namespaces	capabilities	Container host-ns-app has dangerous capability: SYS_ADMIN
low	default/test-7-host-namespaces	read_only_root	Container host-ns-app root filesystem is not read-only

(a) Spec violations (page 3)

(b) Layer 1: Unique spec violations (2/2).

The test suite comprises 44 automated tests organised into two files:

test_all.py (34 tests) covers: the normalizer (9 tests for capability computation including drop-all, drop-specific, add-extra, and no-spec scenarios), the specification extractor (3 tests for single-container, multi-container, and secure-pod extraction), the runtime inspector parser (5 tests covering Docker and crictl JSON parsing for privileged, secure, and seccomp-drift fixtures), the drift detector (13 tests covering all seven comparison dimensions plus integration scenarios), and the policy engine (2 tests for privileged and secure pod detection).

test_evaluation.py (10 tests) mirrors the seven deployment-level test pods, constructing SpecSecurityState and RuntimeSecurityState objects manually to simulate Docker/Minikube behaviour. This includes the Pod 3 vs. Pod 4 controlled comparison test, which verifies that the seccomp declaration is the sole variable producing the CRITICAL finding.

All 44 tests execute in 0.2 seconds without requiring a Kubernetes cluster, enabling rapid development iteration.

4.7 Deployment Procedure

The deploy.sh script automates the complete deployment:

Listing 4.4. Deployment script (simplified)

```
#!/bin/bash
# Build scanner image
docker build -t k8s-psa:latest .
minikube image load k8s-psa:latest

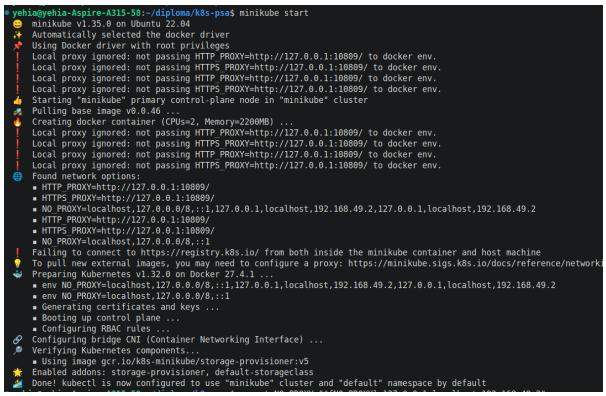
# Build dashboard image
docker build -t k8s-psa-dashboard:latest dashboard/
minikube image load k8s-psa-dashboard:latest

# Deploy all resources
kubectl apply -f deployments/all-in-one.yaml

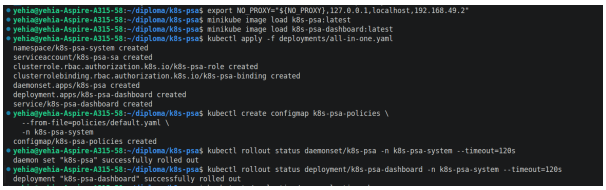
# Wait for pods to be ready
kubectl rollout status daemonset/k8s-psa -n k8s-psa-system
kubectl rollout status deployment/k8s-psa-dashboard -n k8s-psa-system
```

After deployment, the dashboard is accessible via:

```
kubectl port-forward svc/k8s-psa-dashboard 8080:8080 -n k8s-psa-system
# Open http://localhost:8080
```



(a) minikube start output



(b) Image load and deployment

```
yehia@yehia-Aspire-A315-58:~/diploma/k8s-psa$ bash tests/evaluation/run-evaluation.sh
test-2-no-security              1/1     Running   2 (2m54s ago)   33h   10.244.0.16   minikube   <none>   <none>
test-3-privileged-seccomp       1/1     Running   2 (2m54s ago)   33h   10.244.0.12   minikube   <none>   <none>
test-4-privileged-no-seccomp    1/1     Running   2 (2m54s ago)   33h   10.244.0.11   minikube   <none>   <none>
test-5-partial-security         1/1     Running   2 (2m54s ago)   33h   10.244.0.14   minikube   <none>   <none>
test-6-no-privesc               1/1     Running   2 (2m54s ago)   33h   10.244.0.17   minikube   <none>   <none>
test-7-host-namespaces          1/1     Running   2 (31h ago)     33h   192.168.49.2  minikube   <none>   <none>

[4/5] Waiting for 2 scan cycles (130 seconds)...
  The scanner runs every 60s. Waiting for it to detect all test pods.
[5/5] Collecting results...

═══════════════════════════════════════
  LAYER 1: Spec Violations per Test Pod
═══════════════════════════════════════
  Pod 1 (test-1-fully-hardened): 0 unique violations
  Pod 2 (test-2-no-security): 10 unique violations
  Pod 3 (test-3-privileged-seccomp): 10 unique violations
  Pod 4 (test-4-privileged-no-seccomp): 20 unique violations
  Pod 5 (test-5-partial-security): 5 unique violations
  Pod 6 (test-6-no-privesc): 0 unique violations
  Pod 7 (test-7-host-namespaces): 20 unique violations

═══════════════════════════════════════
  LAYER 2: Drift Findings per Test Pod
═══════════════════════════════════════
  Pod 1 (test-1-fully-hardened): CRITICAL=4  WARNING=0  INFO=0
  Pod 2 (test-2-no-security): CRITICAL=0  WARNING=0  INFO=0
  Pod 3 (test-3-privileged-seccomp): CRITICAL=4  WARNING=50  INFO=0
  Pod 4 (test-4-privileged-no-seccomp): CRITICAL=0  WARNING=50  INFO=0
  Pod 5 (test-5-partial-security): CRITICAL=4  WARNING=0  INFO=0
  Pod 6 (test-6-no-privesc): CRITICAL=4  WARNING=0  INFO=0
  Pod 7 (test-7-host-namespaces): CRITICAL=0  WARNING=0  INFO=8

═══════════════════════════════════════
  LAYER 2: Detailed CRITICAL Drift Findings
═══════════════════════════════════════
  [CRITICAL] etcd-minikube/etcd
    Field: seccompProfile
    Declared: RuntimeDefault
    Actual:   unconfined (null)

  [CRITICAL] kube-apiserver-minikube/kube-apiserver
    Field: seccompProfile
    Declared: RuntimeDefault
    Actual:   unconfined (null)

  [CRITICAL] kube-controller-manager-minikube/kube-controller-manager
    Field: seccompProfile
    Declared: RuntimeDefault
    Actual:   unconfined (null)

  [CRITICAL] kube-scheduler-minikube/kube-scheduler
    Field: seccompProfile
    Declared: RuntimeDefault
    Actual:   unconfined (null)

  [CRITICAL] test-1-fully-hardened/secure-app
    Field: seccompProfile
    Declared: RuntimeDefault
    Actual:   unconfined (null)

  [CRITICAL] test-3-privileged-seccomp/priv-seccomp
    Field: seccompProfile
    Declared: RuntimeDefault
    Actual:   unconfined (null)

  [CRITICAL] test-5-partial-security/partial-app
    Field: seccompProfile
```

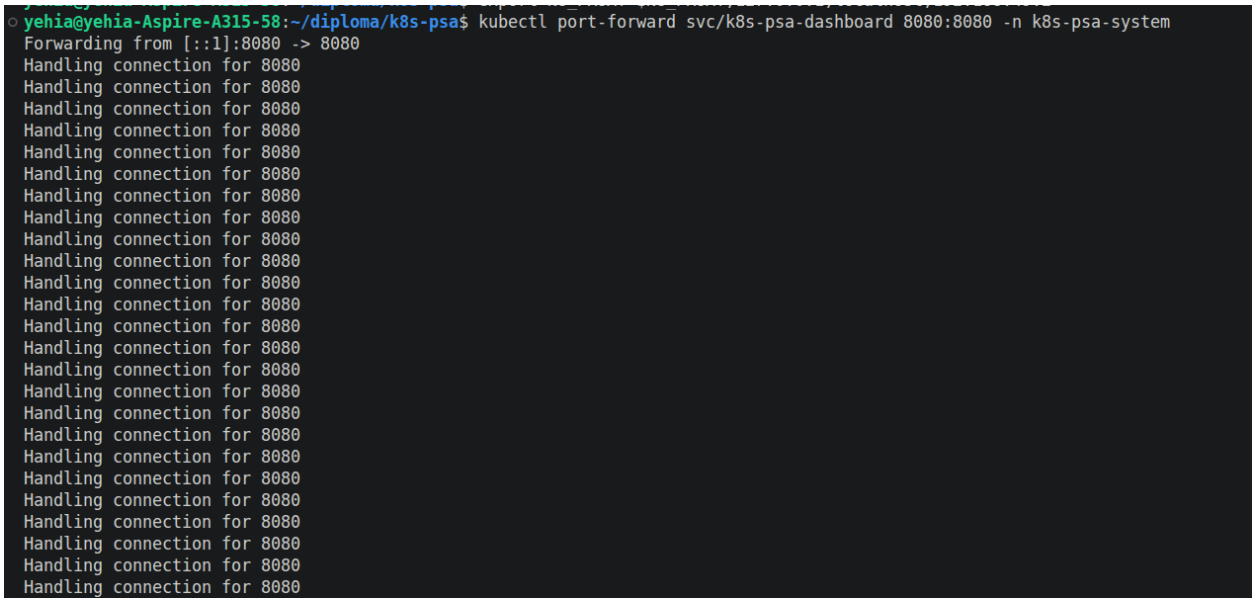


Fig. 4.6. Forwarding the dashboard to the host machine .

Chapter 5

Evaluation and Discussion

With the tool deployed as described in Chapter 4, we now present the experimental results that answer our research questions. The evaluation was conducted on a Minikube cluster with seven purpose-built test pods and four Kubernetes control plane components. We first describe the procedure, then present the detection results, analyse the central finding (privileged seccomp stripping), examine the observability gap, and conclude with a controlled comparison that confirms the tool's specificity.

5.1 Experimental Procedure

The evaluation was conducted on the Minikube cluster described in Section 3.8. The seven test pods (Table 3.3) were deployed using:

```
kubectl apply -f tests/evaluation/pod-1-fully-hardened.yaml
kubectl apply -f tests/evaluation/pod-2-no-security.yaml
kubectl apply -f tests/evaluation/pod-3-privileged-seccomp.yaml
kubectl apply -f tests/evaluation/pod-4-privileged-no-seccomp.yaml
kubectl apply -f tests/evaluation/pod-5-partial-security.yaml
kubectl apply -f tests/evaluation/pod-6-no-privesc.yaml
kubectl apply -f tests/evaluation/pod-7-host-namespaces.yaml
```

The scanner was already running as a DaemonSet. Three complete scan cycles were captured, during which the scanner inspected 9 pods in the first cycle (before test pods started) and 16 pods in subsequent cycles. Results were collected from the scanner's log output.

5.2 Detection Results

5.2.1 Layer 1: Specification Violations

Layer 1 identified specification-level violations across all pods lacking complete security hardening. Table 5.1 summarises the findings.

Table 5.1. Layer 1 specification violations per test pod (unique per pod)

Test Pod	Unique Violations	Failed Policy Checks
1. Fully Hardened	0	None (compliant)
2. No Security Context	2	seccomp_profile, read_only_root
3. Privileged + Seccomp	2	privileged_container, read_only_root
4. Privileged, No Seccomp	4	privileged_container, root_container, seccomp_profile, read_only_root
5. Partial Security	1	read_only_root
6. No Privesc	0	None (compliant)
7. Host Namespaces	6	host_network, host_pid, host_ipc, seccomp_profile, capabilities, read_only_root

These results validate Layer 1 correctness: Pod 1 and Pod 6 are fully hardened and produce zero violations; Pod 7 with three host namespace flags and dangerous capabilities produces the most violations.

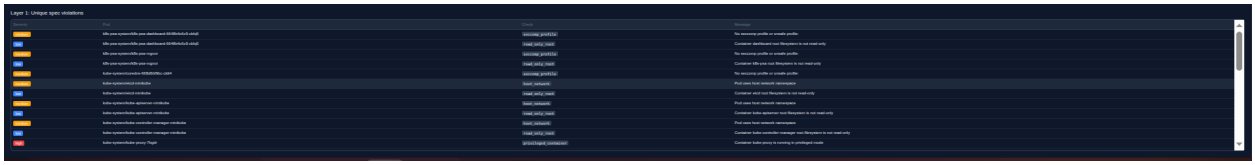


Fig. 5.1. layer1 finding .



Fig. 5.2. layer1 finding .

5.2.2 Layer 2: Drift Findings

Table 5.2 presents the Layer 2 drift findings, which constitute the primary contribution of this work.

Table 5.2. Layer 2 drift findings per test pod (unique per pod)

Test Pod	CRITICAL	WARNING	INFO	Expected
1. Fully Hardened	1*	0	0	0
2. No Security Context	0	0	0	0
3. Privileged + Seccomp	1	27	0	1
4. Privileged, No Seccomp	0	27	0	0
5. Partial Security	1*	0	0	0
6. No Privesc	1*	0	0	0–1
7. Host Namespaces	0	0	2	0

*Observability false positives (Section 5.4)

Table 5.3 presents the findings for Kubernetes system pods.

Table 5.3. Layer 2 drift findings for Kubernetes system pods (unique per pod)

System Pod	CRITICAL	WARNING	INFO
etcd-minikube	1	0	1
kube-apiserver-minikube	1	0	1
kube-controller-manager-minikube	1	0	1
kube-scheduler-minikube	1	0	1
kube-proxy	0	27	1
storage-provisioner	0	0	1



Fig. 5.3. layer2 finding .



Fig. 5.4. layer2 finding .

5.3 True Positive: Privileged Seccomp Stripping

The central finding of this thesis concerns Test Pod 3. This pod declares both privileged: true and seccompProfile.type: RuntimeDefault. The specification instructs the runtime to grant full host access while simultaneously applying system call filtering.

The tool detected a CRITICAL drift finding with the following output:

Listing 5.1. CRITICAL drift finding for Pod 3


```
{
  "field": "seccompProfile",
  "severity": "critical",
  "category": "seccomp_mismatch",
  "description": "Spec declares seccomp type 'RuntimeDefault'
    but the runtime has NO seccomp filter (unconfined)",
  "declared": "RuntimeDefault",
  "actual": "unconfined (null)"
}
```


The Docker runtime applies all 41 Linux capabilities and removes the seccomp filter entirely for privileged containers, even when seccomp is explicitly declared. This behaviour is documented in the Moby project issue tracker [18], where privileged mode overrides all other security settings regardless of user declarations.

This finding has direct security implications. An operator who specifies seccompProfile.type: RuntimeDefault on a privileged container reasonably expects system call filtering to be active. Any specification-only tool — Gatekeeper [5], Kyverno [6], Kubescape, Trivy Operator — reads the Kubernetes API, sees RuntimeDefault, and reports the container as having syscall filtering. Only by querying the container runtime does the silent removal become visible.

Additionally, the tool detected 27 WARNING-level capability findings for Pod 3. The normalizer computes the expected set as the 14 defaults (since no explicit capabilities.add is declared), but the runtime grants all 41 capabilities due to privileged mode. Each of the 27 non-default capabilities — including CAP_SYS_ADMIN, CAP_NET_ADMIN, CAP_SYS_PTRACE, CAP_SYS_MODULE, and CAP_BPF — is flagged as an unexpected addition.

5.4 Observability Gap: Docker's Implicit Seccomp Default

Test Pods 1, 5, and 6 — all non-privileged with seccompProfile.type: RuntimeDefault — unexpectedly produced CRITICAL seccomp drift findings. Investigation revealed that these are *observability false positives* caused by a documented but widely misunderstood behaviour of the Docker runtime metadata interface.

When the CRI adapter (cri-dockerd) translates Kubernetes RuntimeDefault into Docker API calls, it does not pass an explicit SecurityOpt entry [16]. Instead, it relies on Docker Engine's implicit default behaviour: when no seccomp option is specified, Docker applies its built-in profile automatically. According to the Docker documentation [16], this built-in profile operates as an allowlist that restricts several dozen sensitive system calls, among them

mount, ptrace, kexec_load, and reboot [15].

Critically, Docker's implicit default profile does not appear in docker inspect output. The SecurityOpt field shows null or an empty array, which is indistinguishable from a genuinely unprotected container. The CIS Docker Benchmark (check 5.21) documents this: an empty SecurityOpt indicates the *default profile is active*, not that seccomp is disabled [24], [35].

The four Kubernetes control plane pods (etcd, kube-apiserver, kube-controller-manager, kube-scheduler) exhibit the same phenomenon. These pods declare seccompProfile.type: RuntimeDefault and use hostNetwork: true, yet Docker's inspection metadata shows no seccomp configuration. This occurs within Minikube's Docker-in-Docker architecture, where the outer node container operates with –privileged –security-opt seccomp=unconfined [36].

This finding reveals that the container runtime metadata interface is not a transparent window into kernel-level enforcement — it is a lossy abstraction that omits information about implicit defaults. A definitive verification would require reading the kernel's seccomp enforcement status from /proc/<pid>/status, where the Seccomp field value of 2 confirms BPF filter mode is active [17].

5.5 Pod 3 vs. Pod 4: The Controlled Comparison

The comparison between Pods 3 and 4 provides the most rigorous evidence for the enforcement verification gap. Table 5.4 presents the side-by-side comparison.

Table 5.4. Pod 3 vs. Pod 4: controlled comparison isolating the seccomp declaration variable

Property	Pod 3	Pod 4
Spec: privileged	true	true
Spec: seccompProfile	RuntimeDefault	Not declared
Runtime: capabilities	All 41	All 41
Runtime: seccomp	Unconfined	Unconfined
Runtime: masked paths	None	None
Layer 2 CRITICAL	**1** (seccomp mismatch)	**0**
Layer 2 WARNING	27 (capabilities)	27 (capabilities)

Both pods have *identical* runtime states: all 41 capabilities, no seccomp filter, no masked paths. The only difference is the seccomp declaration in Pod 3's Kubernetes specification. The drift detector correctly identifies Pod 3 as having a CRITICAL seccomp mismatch (declared restriction not enforced) and Pod 4 as having no seccomp mismatch (no declaration to violate).

This controlled comparison eliminates confounding variables and demonstrates that the tool's *specificity* — its ability to distinguish between "restriction declared but not enforced" and "restriction never declared" — is functioning correctly.

5.6 Performance

Table 5.5 summarises the performance characteristics observed during the evaluation.

Table 5.5. Performance characteristics of k8s-psa

Metric	Value
Scan cycle duration (16 pods)	≈3.1 seconds
Layer 1 duration	≈0.7 seconds
Layer 2 duration (18 containers)	≈2.4 seconds
Scanner memory usage	64 MiB
Scanner CPU usage	<100m
Dashboard memory usage	32 MiB
Unit test execution time (44 tests)	0.2 seconds

The scan cycle duration of 3.1 seconds is well within the 60-second scan interval, leaving ample headroom for larger clusters. The resource consumption is modest enough for continuous operation alongside production workloads.

5.7 Discussion

5.7.1 Answering the Research Questions

RQ1: To what extent does divergence exist? The results demonstrate that divergence between declared and enforced security is not hypothetical. Privileged containers with seccomp declarations exhibit a genuine enforcement gap where Docker silently removes the declared restriction. Additionally, Docker's metadata interface creates an observability gap that makes correctly protected containers appear unprotected.

RQ2: Can a lightweight tool detect such divergences? Yes. k8s-psa correctly identified the seccomp stripping in Pod 3 as CRITICAL, correctly produced zero CRITICAL findings for the Pod 4 control, correctly flagged 27 capability drifts per privileged pod, and correctly classified namespace mode discrepancies as INFO. The tool operates within 64 MiB memory and completes scan cycles in 3.1 seconds.

RQ3: What categories of divergence exist? Three categories emerged from the evaluation:

1. **Genuine enforcement gap** (CRITICAL): The seccomp profile is declared but stripped by the runtime's privileged mode implementation. This is a true security deficiency.
2. **Capability inflation** (WARNING): Privileged mode grants all 41 capabilities, 27 beyond what was explicitly requested. This is expected but worth flagging.
3. **Observability gap** (false-positive CRITICAL): Docker's implicit seccomp default does not appear in `docker inspect` output, causing correctly protected containers to appear unprotected.

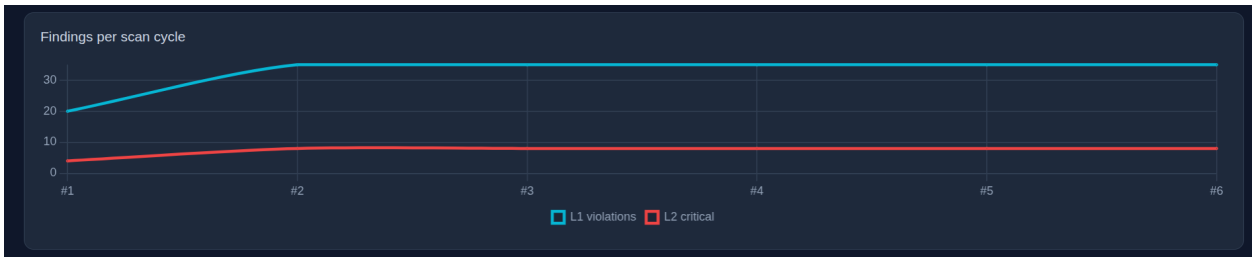


Fig. 5.5. divergence grapgh .

5.7.2 Comparison with Existing Tools

No existing open-source tool performs the specification-versus-runtime comparison that k8s-psa implements. Table 5.6 positions the tool relative to the existing landscape.

Table 5.6. Comparison of k8s-psa with existing tools

Tool	What It Checks	Layer
OPA Gatekeeper	Spec vs. policy constraints	Kubernetes API only
Kyverno	Spec vs. validate/mutate rules	Kubernetes API only
Falco	System call behaviour (eBPF)	Kernel only
Kubediff	Spec vs. git manifests	API vs. git
k8s-psa	**Spec vs. actual runtime state**	**API vs. runtime**

5.8 Limitations

This evaluation has several limitations:

1. **Single runtime tested.** The experiments use Minikube with Docker. Production clusters predominantly use containerd, where the seccomp observability gap may not exist because `crictl inspect` includes the full OCI runtime specification with explicit seccomp configuration.

2. **Metadata-based verification.** The seccomp check relies on `docker inspect` metadata rather than kernel-level enforcement status from `/proc/<pid>/status`. A definitive check would verify the `Seccomp` field (value 2 confirms BPF filter mode).

3. **Single-node cluster.** Production clusters involve multi-node architectures, network policies, and service meshes that may affect security context translation.

4. **CLI-based inspection.** The tool shells out to `kubectl` and `docker` commands rather than using native API client libraries, introducing parsing fragility.

5. **Limited test pod count.** Seven test pods cover the primary security dimensions but do not exhaust all possible configurations (e.g., init containers, sidecar containers, or ephemeral containers).

5.8.1 Future Work

Three directions merit investigation. First, extending the drift detector with $/\mathrm{proc}/<\mathrm{pid}>/\mathrm{status}$ verification would eliminate the seccomp observability gap entirely by reading the kernel's enforcement status directly. Second, deploying the tool on containerd-based clusters (kind, GKE, EKS) would enable comparison of the enforcement landscape across runtime implementations. Third, investigating whether mutating admission webhooks create additional enforcement gaps — a webhook could modify security contexts after user submission, introducing discrepancies before the pod ever reaches the runtime.

Chapter 6

Conclusion

This thesis set out to bridge the enforcement verification gap — a blind spot in the Kubernetes security ecosystem where no tool verifies that declared security contexts are actually enforced. By designing, implementing, and evaluating k8s-psa, we have not only detected a genuine enforcement gap (seccomp stripping in privileged containers) but also exposed an observability gap in Docker's metadata interface. We now summarise the findings, restate the contributions, and outline the practical implications.

6.1 Summary of Findings

This thesis identified and addressed the enforcement verification gap in Kubernetes container security — the absence of tooling that verifies whether declared security contexts are faithfully enforced by the container runtime. We designed, implemented, and evaluated k8s-psa, a DaemonSet-based tool that continuously compares Kubernetes API specifications against Docker runtime state across seven security dimensions.

Through controlled experiments on a Minikube cluster with seven purpose-built test pods and four Kubernetes control plane components, we arrived at three principal findings:

Finding 1: The enforcement verification gap is real. Privileged containers that declare $\mathrm{seccompProfile.type:\ RuntimeDefault}$ have that seccomp filter silently stripped by the Docker runtime. This constitutes a genuine enforcement failure where the container runs with fewer restrictions than the operator intended. This finding was confirmed through a controlled comparison between Pod 3 (privileged with seccomp declared, producing 1 CRITICAL finding) and Pod 4 (privileged without seccomp declared, producing 0 CRITICAL findings), isolating the seccomp declaration as the sole variable.

Finding 2: An observability gap complicates runtime inspection. Docker's metadata interface omits information about correctly applied implicit seccomp defaults. When $\mathrm{cri\text{-}dockerd}$ translates Kubernetes $\mathrm{RuntimeDefault}$ into Docker API calls, it relies on Docker's implicit default behaviour without recording an explicit $\mathrm{SecurityOpt}$ entry. This makes properly protected containers indistinguishable from genuinely unprotected containers in $\mathrm{docker\ inspect}$ output, producing false positives in naïve runtime verification.

Finding 3: No existing tool detects this gap. Every tool in the current Kubernetes security ecosystem — OPA Gatekeeper, Kyverno, Kubescape, Trivy Operator, kube-bench [37] — reads security properties from the

Kubernetes API only. None query the container runtime. The enforcement gap identified in Finding 1 is invisible to all of them.

6.2 Contributions Restated

This thesis made four contributions to the field of Kubernetes container security:

1. A novel enforcement verification tool (k8s-psa) that occupies the previously empty space between specification-level admission controllers and kernel-level behavioural monitors. The tool compares declared security intent against actual runtime state across seven dimensions: privileged mode, user identity, capabilities, seccomp profiles, read-only root filesystem, privilege escalation flags, and host namespace sharing.
2. Experimental evidence demonstrating that Docker silently strips declared seccomp profiles from privileged containers. This behaviour is documented in the Moby project issue tracker [18] but had not been detected by any runtime verification tool.
3. Discovery and characterisation of the Docker seccomp observability gap, demonstrating that runtime metadata interfaces are lossy abstractions that omit information about implicit security defaults. This finding has implications for any tool that relies on $\mathrm{docker\ inspect}$ for security posture assessment.
4. An open-source evaluation framework comprising 44 automated unit tests (executable in 0.2 seconds without a cluster), 7 deployment-level test cases with documented expected outcomes, a one-command deployment script, and a real-time Chart.js dashboard for continuous monitoring.

6.3 Practical Implications

The findings of this thesis have three practical implications for Kubernetes security practitioners:

First, operators should be aware that combining $\mathrm{privileged:\ true}$ with $\mathrm{seccompProfile.type:\ RuntimeDefault}$ provides a false sense of security. The seccomp declaration is silently ignored by the Docker runtime. If seccomp filtering is genuinely required, the container should not be privileged; alternatively, the cluster should use containerd, where the seccomp behaviour for privileged containers may differ.

Second, security audit tools that rely solely on the Kubernetes API for compliance verification may report incorrect results for privileged containers. Compliance frameworks such as CIS Kubernetes Benchmark and NSA Kubernetes Hardening Guidance [8], [35] should be supplemented with runtime-level verification.

Third, the Docker runtime metadata interface ($\mathrm{docker\ inspect}$) should not be treated as a definitive indicator of seccomp enforcement. The CIS Docker Benchmark check 5.21 documents that an empty $\mathrm{SecurityOpt}$ indicates the default profile is active, not that seccomp is disabled. Definitive verification requires reading the kernel's enforcement status from $\mathrm{/proc/<pid>/status}$.

6.4 Future Work

Three directions merit further investigation:

1. **Kernel-level seccomp verification.** Extending the drift detector to read /proc/<pid>/status inside each container (via nsenter or hostPID access) would eliminate the seccomp observability gap entirely. A Seccomp field value of 2 confirms that BPF filter mode is active, regardless of what docker inspect reports.

2. **Cross-runtime evaluation.** Deploying k8s-psa on containerd-based clusters (kind, GKE, EKS) and CRI-O-based clusters (OpenShift) would enable comparison of the enforcement verification landscape across different runtime implementations. The seccomp observability gap may not exist in containerd, where crictl inspect includes the full OCI runtime specification.

3. **Mutating webhook drift.** Investigating whether mutating admission webhooks create additional enforcement gaps represents a distinct research direction. A mutating webhook could alter security contexts after user submission — for instance, adding a seccomp profile — without the user's knowledge, creating discrepancies that persist even when the runtime functions correctly.

Bibliography cited

[1] A. Y. D. Wong, E. G. Chekole, M. Ochoa, and J. Zhou, "On the security of containers: Threat modeling, attack analysis, and mitigation strategies," *Computers & Security*, vol. 128, p. 103 140, May 2023. doi: 10.1016/j.cose.2023.103140.

[2] Cloud Native Computing Foundation, *CNCF annual survey 2021: The year Kubernetes crossed the chasm*, [Online]. Available: https://www.cncf.io/reports/cncf-annual-survey-2021/, Feb. 2022.

[3] S. Sultan, I. Ahmad, and T. Dimitriou, "Container security: Issues, challenges, and the road ahead," *IEEE Access*, vol. 7, pp. 52 976–52 996, 2019. doi: 10.1109/ACCESS.2019.2911732.

[4] T. Combe, A. Martin, and R. Di Pietro, "To Docker or not to Docker: A security perspective," *IEEE Cloud Computing*, vol. 3, no. 5, pp. 54–62, 2016. doi: 10.1109/MCC.2016.100.

[5] Open Policy Agent, *Gatekeeper audit*, [Online]. Available: https://open-policy-agent.github.io/gatekeeper/website/docs/audit/, 2024.

[6] Kyverno, *Background scans*, [Online]. Available: https://kyverno.io/docs/policy-reports/background/, 2024.

[7] Kubernetes, *Kubernetes v1.25: Pod Security Admission controller in stable*, [Online]. Available: https://kubernetes.io/blog/2022/08/25/pod-security-admission-stable/, Aug. 2022.

[8] National Security Agency and Cybersecurity and Infrastructure Security Agency, *Kubernetes hardening guide*, Cybersecurity Technical Report, v1.2. [Online]. Available: https://media.defense.gov/2022/Aug/29/2003066362/-1/-1/0/CTR_KUBERNETES_HARDENING_GUIDANCE_1.2_20220829.PDF, Aug. 2022.

[9] Kubernetes, *Restrict a container's syscalls with seccomp*, [Online]. Available: https://kubernetes.io/docs/tutorials/security/seccomp/, 2024.

[10] T. Bui, *Analysis of Docker security*, arXiv preprint arXiv:1501.02967. [Online]. Available: https://arxiv.org/abs/1501.02967, 2015.

[11] X. Lin, L. Lei, Y. Wang, J. Jing, K. Sun, and Q. Zhou, "A measurement study on Linux container security: Attacks and countermeasures," in *Proceedings of the 34th Annual Computer Security Applications Conference (ACSAC '18)*, San Juan, PR, USA, 2018, pp. 418–429. doi: 10.1145/3274694.3274720.

[12] Kubernetes, *Pod security standards*, [Online]. Available: https://kubernetes.io/docs/concepts/security/pod-security-standards/, 2024.

[13] Open Container Initiative, *OCI runtime specification*, GitHub. [Online]. Available: https://github.com/opencontainers/runtime-spec/blob/master/config.md, 2023.

[14] M. Kerrisk, *Capabilities(7) — overview of Linux capabilities*, Linux man-pages project. [Online]. Available: https://man7.org/linux/man-pages/man7/capabilities.7.html, 2024.

[15] R. McCune, *Container security fundamentals part 6: Seccomp*, Datadog Security Labs. [Online]. Available: https://securitylabs.datadoghq.com/articles/container-security-fundamentals-part-6/, Sep. 2023.

[16] Docker, *Seccomp security profiles for Docker*, Docker Docs. [Online]. Available: https://docs.docker.com/engine/security/seccomp/, 2024.

[17] S. Grunert and M. Patel, *KEP-2413: Enable seccomp by default*, Kubernetes Enhancement Proposal, GA in Kubernetes v1.27. [Online]. Available: https://github.com/kubernetes/enhancements/tree/master/keps/sig-node/2413-seccomp-by-default, 2023.

[18] Moby, *Allow for custom seccomp profiles to be used in privileged containers*, GitHub, Issue #41321. [Online]. Available: https://github.com/moby/moby/issues/41321, 2020.

[19] A. Rahman, M. S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source Kubernetes manifests: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–36, May 2023. doi: 10.1145/3579639.

[20] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, *GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation*, arXiv preprint arXiv:2405.19954 [cs.CR]. [Online]. Available: https://arxiv.org/abs/2405.19954, May 2024.

[21] O. Sgan Cohen, E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, *KubeGuard: LLM-assisted Kubernetes hardening via configuration files and runtime logs analysis*, arXiv preprint arXiv:2509.04191 [cs.CR]. [Online]. Available: https://arxiv.org/abs/2509.04191, Sep. 2025.

[22] Sysdig, *Container drift detection with Falco*, [Online]. Available: https://www.sysdig.com/blog/container-drift-detection-with-falco, 2024.

[23] Y. He, R. Guo, Y. Xing, *et al.*, "Cross container attacks: The bewildered eBPF on clouds," in *32nd USENIX Security Symposium (USENIX Security '23)*, Anaheim, CA, USA, 2023, pp. 5971–5988.

[24] Z. Morić, V. Dakić, and T. Čavala, "Security hardening and compliance assessment of Kubernetes control plane and workloads," *Journal of Cybersecurity and Privacy*, vol. 5, no. 2, p. 30, Jun. 2025. doi: 10.3390/jcp5020030.

[25] Weaveworks, *Kubediff*, GitHub. [Online]. Available: https://github.com/weaveworks/kubediff, 2020.

[26] Kubernetes, *Using admission controllers to detect container drift at runtime*, [Online]. Available: https://kubernetes.io/blog/2021/12/21/admission-controllers-for-container-drift/, Dec. 2021.

[27] Komodor, *Drift detection in Kubernetes*, [Online]. Available: https://komodor.com/blog/drift-detection-in-kubernetes/, 2023.

[28] H. Kermabon-Bobinnec, M. Gholipourchoubeh, S. Bagheri, *et al.*, "ProSPEC: Proactive security policy enforcement for containers," in *Proceedings of the 12th ACM Conference on Data and Application Security and Privacy (CODASPY '22)*, Baltimore, MD, USA, Apr. 2022, pp. 1–12. doi: 10.1145/3508398.3511515.

[29] M. S. I. Shamim, H. Hu, and A. Rahman, "Dynamic application security testing for Kubernetes deployment: An experience report from industry," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE '25), Industry Papers Track*, Trondheim, Norway, Jun. 2025. doi: 10.1145/3696630.3728573.

[30] Devi Priya V S, S. C. Sethuraman, and M. K. Khan, "Container security: Precaution levels, mitigation strategies, and research perspectives," *Computers & Security*, vol. 135, p. 103 490, Dec. 2023. doi: 10.1016/j.cose.2023.103490.

[31] CloudSecBurrito, *Kubernetes securityContext explained: Intent, not enforcement*, [Online]. Available: https://cloudsecburrito.com/kubernetes-securitycontext, 2025.

[32] R. Shu, X. Gu, and W. Enck, "A study of security vulnerabilities on Docker Hub," in *Proceedings of the Seventh ACM Conference on Data and Appli-*

cation Security and Privacy (CODASPY '17), Scottsdale, AZ, USA, 2017, pp. 269–280. doi: 10.1145/3029806.3029832.

[33] Trail of Bits and Atredis Partners, *Kubernetes security assessment*, Commissioned by the CNCF. [Online]. Available: https://github.com/trailofbits/audit-kubernetes/blob/master/reports/Kubernetes%20Security%20Review.pdf, May 2019.

[34] Kubernetes, *Daemonset*, [Online]. Available: https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/, 2024.

[35] Center for Internet Security, *CIS Docker benchmark*, v1.7.0. [Online]. Available: https://www.cisecurity.org/benchmark/docker, 2024.

[36] Kubernetes, *Kic: Run docker container without –privileged*, GitHub, kubernetes/minikube Issue #6387. [Online]. Available: https://github.com/kubernetes/minikube/issues/6387, 2020.

[37] Aqua Security, *Kube-bench*, GitHub. [Online]. Available: https://github.com/aquasecurity/kube-bench, 2024.

Appendices

Appendix A: Tool Source Code Structure

The complete source code of k8s-psa comprises 37 files organised as follows:

Listing 6.1. k8s-psa project structure

```
k8s-psa/
  src/
    models.py              # SpecSecurityState, RuntimeSecurityState,
                           # DriftFinding, DriftReport dataclasses
    normalizer.py          # 14 default capabilities, compute_expected_capabilities()
    pod_scanner.py         # kubectl get pods --all-namespaces -o json
    spec_extractor.py      # Pod JSON -> SpecSecurityState
    runtime_inspector.py   # docker inspect / crictl inspect -> RuntimeSecurityState
    policy_engine.py       # Layer 1: 8 configurable spec-level checks
    drift_detector.py      # Layer 2: 7 field-by-field comparisons
    alert_manager.py       # NDJSON output to /var/log/k8s-psa/
    main.py                # DaemonSet entry point, scan loop orchestrator
  dashboard/
    server.py              # Zero-dependency HTTP server, Chart.js UI
    Dockerfile             # Dashboard container image
  tests/
    test_all.py            # 34 core unit tests
    test_evaluation.py     # 10 evaluation scenario unit tests
    fixtures/              # Docker + crictl inspect JSON samples
      docker_inspect_privileged.json
      docker_inspect_secure.json
      crictl_inspect_privileged.json
```

```
    crictl_inspect_secure.json
    crictl_inspect_seccomp_drift.json
  evaluation/            # 7 test pod YAML manifests
    pod-1-fully-hardened.yaml
    pod-2-no-security.yaml
    pod-3-privileged-seccomp.yaml
    pod-4-privileged-no-seccomp.yaml
    pod-5-partial-security.yaml
    pod-6-no-privesc.yaml
    pod-7-host-namespaces.yaml
    run-evaluation.sh
    TEST-EXPECTATIONS.md
deployments/
  all-in-one.yaml        # Namespace, ClusterRole, ClusterRoleBinding,
                         # DaemonSet, Deployment, Service
policies/
  default.yaml           # Configurable policy thresholds
Dockerfile               # Scanner container image
deploy.sh                # One-command deployment script
analyze.py               # Offline log analysis + static dashboard
requirements.txt         # Python dependencies
```

Appendix B: Evaluation Test Pod Specifications

B.1 Pod 1: Fully Hardened Baseline

Listing 6.2. Pod 1 — fully hardened baseline (false-positive control)

```
apiVersion: v1
kind: Pod
metadata:
  name: test-1-fully-hardened
spec:
  securityContext:
```

```
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: secure-app
    image: busybox:latest
    command: ["sleep", "3600"]
    securityContext:
      runAsUser: 1000
      runAsGroup: 1000
      readOnlyRootFilesystem: true
      allowPrivilegeEscalation: false
      capabilities:
        drop: ["ALL"]
```

B.2 Pod 3: Privileged + Seccomp Declared (Key Scenario)

Listing 6.3. Pod 3 — privileged container with seccomp declared

```
apiVersion: v1
kind: Pod
metadata:
  name: test-3-privileged-seccomp
spec:
  securityContext:
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: priv-seccomp
    image: busybox:latest
    command: ["sleep", "3600"]
    securityContext:
      privileged: true
```

B.3 Pod 4: Privileged Without Seccomp (Scientific Control)

Listing 6.4. Pod 4 — privileged container without seccomp (control for Pod 3)

```
apiVersion: v1
kind: Pod
metadata:
  name: test-4-privileged-no-seccomp
spec:
  containers:
  - name: priv-no-seccomp
    image: busybox:latest
    command: ["sleep", "3600"]
    securityContext:
      privileged: true
      runAsUser: 0
```

Appendix C: Real-Time Dashboard

The k8s-psa real-time dashboard is deployed as a Kubernetes Deployment and accessed via port forwarding:

```
kubectl port-forward svc/k8s-psa-dashboard 8080:8080 -n k8s-psa-system
```

The dashboard provides six visualisation panels:

1. **Drift findings by severity** (doughnut chart): Shows the distribution of CRITICAL, WARNING, and INFO findings across all scanned containers.
2. **Drift findings by security field** (horizontal bar chart): Breaks down findings by the seven security dimensions (seccompProfile, capabilities, hostNetwork, etc.).
3. **Drift findings by category** (horizontal bar chart): Groups findings by their semantic category (seccomp_mismatch, unexpected_addition, benign_default, etc.).
4. **Specification violations by check** (horizontal bar chart): Shows Layer 1 findings by policy check type (privileged_container, seccomp_profile, read_only_root, etc.).
5. **Findings per scan cycle** (time-series line chart): Tracks the number of findings over time to visualise how the security posture evolves.
6. **Detailed findings tables**: Scrollable tables listing all unique drift findings and specification violations, sorted by severity, with declared and actual values shown side by side.

The dashboard frontend polls /api/stats every 10 seconds. The server reads the NDJSON log files, aggregates findings, and returns a JSON statistics object. The saved HTML snapshot of the running dashboard is provided alongside this thesis.

Appendix D: Experimental Environment Details

Table 6.1. Experimental environment configuration

Component	**Version / Configuration**
Host operating system	Ubuntu 22.04 LTS
Minikube	v1.35.0, Docker driver
Kubernetes	v1.32.0
Container runtime	Docker Engine 27.4.1 (via cri-dockerd)
Scanner language	Python 3.12
Scanner scan interval	60 seconds
Test container image	busybox:latest
Test container command	sleep 3600
Data collection	3 complete scan cycles
Scanner resource limits	100m CPU, 128 MiB memory
Dashboard resource limits	50m CPU, 64 MiB memory